



Számítógép építő többbrétegű alkalmazás

Készítette

Kovács Erik

Programtervező Informatikus BSc

Témavezető

Dr. Biró Csaba

Egyetemi docens

EGER, 2025

Tartalomjegyzék

1. Bevezetés	4
Bevezetés	4
1.1. A projekt célja és a motiváció	4
1.2. Felhasznált technológiák	5
1.3. Miért ezeket a technológiákat választottam?	5
1.3.1. Miért Node.js?	5
1.3.2. Miért Express.js?	6
1.3.3. Miért MySQL és PHPMyAdmin?	6
1.3.4. Miért REST API?	7
1.3.5. Frontend: Miért C# és WPF?	7
2. Specifikáció	8
2.1. Rendszeráttekintés	8
2.2. Funkcionális követelmények	8
2.3. Nem funkcionális követelmények	9
2.4. Adatmodell és adattárolás	9
2.5. Tervezési minták	10
2.5.1. MVVM (Model-View-ViewModel) tervezési minta	10
2.5.2. Visitor tervezési minta az alkatrészek kompatibilitásának ellen- őrzésére	11
2.6. Használati esetek (Use-case-ek)	12
2.7. Verziókezelés	13
2.8. Korlátozások és jövőbeli fejlesztési lehetőségek	13
3. Fejlesztési folyamat és rendszerfelépítés	14
3.1. Fejlesztési stratégia	14
3.2. Az adatbázis 1.0	15
3.3. Az adatbázis 1.0 hibái	15
3.4. Az adatbázis 2.0	19
3.4.1. A processzor és melléktáblái	19
3.4.2. A processzor hűtő és melléktáblái	22

3.4.3.	A grafikus kártya és melléktáblái	25
3.4.4.	Az alaplap és melléktáblái	28
3.4.5.	1:N kapcsolatú melléktáblák	30
3.4.6.	A tápegység és melléktáblái	31
3.4.7.	A RAM és melléktáblái	33
3.4.8.	A ROM és melléktáblái	34
3.5.	A szerver	35
3.5.1.	Szerverindító fájl – server.js	35
3.5.2.	Csomagkezelés – package.json és package-lock.json	36
3.5.3.	Az alkalmazás belső motorja – app.js	36
3.5.4.	Middleware-ek beállítása	37
3.5.5.	Útvonalak (route-ok) regisztrálása	37
3.5.6.	Útvonalak kezelése - Példa: motherboards.js	37
3.5.7.	JSON válasz	39
3.5.8.	Hibakezelés	39
3.6.	A kliens	39
3.6.1.	A főoldal	40
3.6.2.	Listázásra használt nézet. Példa: MotherboardView	43
3.6.3.	Kiválasztott alkatrész részletes megjelenítése. Példa: Selected-MotherboardView	48
3.6.4.	Kiválasztott alkatrész ViewModel-je. Példa: SelectedMotherboardViewModel	49
3.6.5.	Segédablak	50
3.6.6.	Teszt funkció	52
3.6.7.	Teszt funkció	53
3.6.8.	Az eredményt kimutató ablak	55
4.	Tesztelés	58
4.1.	Manuális tesztelés	58
4.1.1.	Tesztelési jegyzőkönyv - Főablak	59
4.1.2.	Tesztelési jegyzőkönyv - Lista nézet	63
4.1.3.	Tesztelési jegyzőkönyv - Részletező ablak	67
4.1.4.	Tesztelési jegyzőkönyv - Részletező ablak	70
4.1.5.	Tesztelési jegyzőkönyv - Teszt funkció	73
4.1.6.	Tesztelési jegyzőkönyv - Eredményt jelző felugró ablak	76
4.2.	Automatizált tesztelés	79
5.	Befejezés	80

1. fejezet

Bevezetés

A modern technológia fejlődésével egyre nagyobb igény mutatkozik a számítógépek testreszabására, legyen szó otthoni, munkahelyi vagy gamer konfigurációkról. A számítógép-összeállítás azonban nem csupán az alkatrészek beszerzéséről szól, hanem azok kompatibilitásának ellenőrzéséről is. Egy nem megfelelően kiválasztott processzor vagy alaplap könnyen használhatatlanná teheti a rendszert, amely nemcsak anyagi veszteséget jelenthet, hanem idővesztést is.

Ennek a problémának a megoldására fejlesztettem egy számítógép-összeállító és kompatibilitás-ellenőrző alkalmazást, amely segít a felhasználóknak az alkatrészek kiválasztásában és azok kompatibilitásának ellenőrzésében. Az alkalmazás egy C# alapú WPF kliens, amely MVVM architektúrát követ, és egy Node.js szerveren keresztül kommunikál egy PHPMyAdmin-ban kezelt adatbázissal. Az alkalmazás célja, hogy a felhasználók könnyedén és gyorsan összeállíthassanak egy működőképes számítógépet, miközben az esetleges hibákról részletes visszajelzést kapnak.

A fejlesztés célja egy olyan rendszer létrehozása volt, amely felhasználóbarát módon segíti a PC-építés folyamatát. Az alkalmazás fő funkciói közé tartozik a gyors adatlekérés, a valós idejű kompatibilitási ellenőrzés és az intuitív felhasználói felület.

A dolgozat további fejezeteiben részletesen bemutatom az egyes komponenseket, a fejlesztési folyamatot, valamint az alkalmazás működését és tesztelését.

1.1. A projekt célja és a motivációm

A projekt létrehozásának egyik legfőbb motivációja az volt, hogy egy olyan eszközt hozzak létre, amely leegyszerűsíti a PC-építés folyamatát és segít a kompatibilitási hibák kiszűrésében. Számos olyan online eszköz létezik, amely lehetőséget biztosít az alkatrészek kiválasztására, azonban ezek gyakran nem adnak visszajelzést arról, hogy egy adott konfiguráció működőképes lesz-e vagy sem. Az általam fejlesztett alkalmazás ezzel szemben részletes elemzést végez az összes kiválasztott alkatrész kompatibilitásáról, és ha probléma merül fel, akkor pontosan meghatározza annak okát és értesíti a

felhasználót.

A saját tapasztalataim is ösztönöztek a projekt elkészítésére. Számos alkalommal találkoztam olyan esettel, amikor egy számítógép-összeállítás során kompatibilitási problémákba ütköztek az informatikában nem igazán jártas ismerőseim. Ezek a hibák sokszor csak utólag derültek ki, amikor az alkatrészek már megvásárlásra kerültek. Ez arra sarkallt, hogy egy olyan alkalmazást készítsek, amely segít megelőzni ezeket a hibákat, és a felhasználóknak pontos visszajelzést nyújt arról, hogy az általuk összeállított konfiguráció működőképes-e.

1.2. Felhasznált technológiák

A projekt során különböző modern technológiákat alkalmaztam, hogy biztosítsam az alkalmazás hatékony működését:

- Backend: Node.js + Express.js
- Adatbázis-kezelés: MySQL (PHPMyAdmin segítségével)
- Frontend (kliens): C# + WPF
- Kommunikáció: REST API

Ezek a technológiák együtt egy rugalmas, gyors és könnyen skálázható rendszert eredményeznek, amely lehetővé teszi a kliens és a szerver közötti zökkenőmentes adatcserét.

1.3. Miért ezeket a technológiákat választottam?

A projekt megvalósítása során olyan technológiákat választottam, amelyek biztosítják a gyors fejlesztést, a hatékony működést és a könnyű karbantarthatóságot. Az alábbiakban bemutatom a választásom indokait, valamint összehasonlítom őket más elérhető lehetőségekkel.

1.3.1. Miért Node.js?

A backend fejlesztéséhez a Node.js környezetet választottam, mivel egy modern, aszinkron és eseményvezérelt serveroldali JavaScript futtatókörnyezet. A Node.js kiválóan alkalmas nagy terhelésű, valós idejű alkalmazásokhoz és REST API-k készítésére, mivel nem blokkoló (non-blocking) I/O modellre épül[1].

Másik lehetőségként használhattam volna például a PHP-t vagy a Java Spring Bootot. A PHP népszerű választás a webfejlesztésben, azonban alapvetően szinkron módon

működik, ami teljesítménybeli hátrányokat jelenthet egy gyors és skálázható API esetében. A Java Spring Boot robusztus és széles körben használt keretrendszer, azonban a fejlesztési sebessége és erőforrásigénye miatt kevésbé praktikus egy kisebb méretű projektnél.

A Node.js nagy előnye, hogy modern, gyors és aszinkron működésű szerverkörnyezet, amely ideális REST API-k létrehozására. Az npm (Node Package Manager) segítségével rengeteg csomag áll rendelkezésre, amelyek gyorsítják a fejlesztést. Emellett széles körben elterjedt és nagy közösségi támogatással rendelkezik.

1.3.2. Miért Express.js?

A Node.js önmagában egy futtatókönyezet, nem pedig egy teljes keretrendszer, ezért szükség volt egy olyan megoldásra, amely egyszerűsíti és felgyorsítja a backend fejlesztését. Erre a célra az Express.js-t választottam, amely egy könnyű, minimalista és rendkívül népszerű webes keretrendszer[2].

Alternatívaként választhattam volna NestJS-t, amely szintén Node.js alapú, de strukturáltabb, modulárisabb megközelítést alkalmaz, és erősen inspirálódik az Angular architektúrájából. Bár a NestJS kiváló választás komplexebb backend rendszerekhez, az én projektem esetében az Express.js egyszerűsége és rugalmassága megfelelőbb volt.

Egy másik opció a Koa.js lehetett volna, amely szintén egy minimalista Node.js keretrendszer, de kevesebb beépített funkcióval rendelkezik, mint az Express.js, így több külső csomagra lett volna szükség. Az Express.js iparági szabványnak számít, rengeteg dokumentáció és támogatás áll rendelkezésre hozzá, ezért ezt választottam.

1.3.3. Miért MySQL és PHPMyAdmin?

Az adatbáziskezeléshez a MySQL-t használtam, mivel egy megbízható, skálázható és széles körben támogatott relációs adatbázis-kezelő rendszer. A PHPMyAdmin egy grafikus felületet biztosít a MySQL kezeléséhez, ami megkönnyíti az adatbázis adminisztrációját.

Alternatívaként választhattam volna PostgreSQL-t, amely fejlettebb tranzakciókezelést és komplexebb lekérdezési lehetőségeket kínál, azonban a MySQL gyorsabb és egyszerűbb megoldás volt a projekt szempontjából. Egy másik lehetőség a MongoDB lett volna, amely egy NoSQL adatbázis, és jobban illeszkedik dinamikusabb, dokumentumalapú adatszerkezetekhez. Mivel az én projektemben az adatok erősen strukturáltak (például alkatrészek, specifikációk), a relációs adatbázisok előnyeit kihasználva [3] a MySQL mellett döntöttem.

1.3.4. Miért REST API?

A szerver és a kliens közötti kommunikációra a REST API-t választottam, mivel egyszerű, jól dokumentált és széles körben támogatott szabvány. Illetve a REST API-k legnagyobb előnye, hogy platformfüggetlenek [4], könnyen bővíthetők, és jól működnek JSON-alapú adatátvitellel, ami a kliens oldalon később nagyon sokat segített.

Másik lehetőségként használhattam volna GraphQL-t, amely nagyobb rugalmasságot biztosít az adatlekérdezések során, mivel a kliens pontosan meghatározhatja, milyen adatokat szeretne kapni. Ugyanakkor a REST API egyszerűbb és kevesebb konfigurációt igényel, így egy viszonylag kisebb projekt számára praktikusabb megoldásnak bizonyult.

1.3.5. Frontend: Miért C# és WPF?

A felhasználói felület fejlesztésére a C# nyelvet és a WPF (Windows Presentation Foundation) keretrendszert választottam. A C# egy erős típusos nyelv [5], amely gyors, biztonságos és jól támogatott a Microsoft ökoszisztémájában. A WPF lehetővé teszi a modern, rezponzív és esztétikus UI kialakítását, amely jól illeszkedik az asztali alkalmazásokhoz [16].

Alternatívaként választható lett volna például az Electron.js-t, amely JavaScript-alapú, és lehetővé teszi a webtechnológiák (HTML, CSS, JavaScript) használatát asztali alkalmazásokhoz. Bár az Electron.js platformfüggetlen, nagy hátránya a magas memórialhasználás és az, hogy egy teljes böngészőt futtat minden egyes alkalmazás esetében. A WPF ezzel szemben natív Windows-támogatást biztosít, ami jobb teljesítményt és kisebb erőforrásigényt eredményez [6].

Egy másik lehetőség a JavaFX lett volna, amely Java-alapú, és szintén támogatja a modern UI fejlesztést. Azonban a WPF jobban integrálható a Windows környezettel, valamint erősebb eszközkészlettel rendelkezik a Microsoft platformokon, ami megint csak a fejlesztés gyorsaságát növeli és a mai világban ez az egyik kulcsa a sikerességnek.

2. fejezet

Specifikáció

2.1. Rendszeráttekintés

A fejlesztett alkalmazás egy PC konfigurátor és kompatibilitás-ellenőrző rendszer, amely lehetővé teszi a felhasználók számára, hogy számítógép-alkatrészeket válasszanak, és ellenőrizzék azok kompatibilitását.

A rendszer alapvetően egy háromrétegű architektúrán alapul:

1. Frontend (kliensoldal) – Egy C# WPF alkalmazás, amely a felhasználók számára biztosít egy grafikus kezelőfelületet az alkatrészek kiválasztásához és az ellenőrzéshez.
2. Backend (szerveroldal) – Egy Node.js + Express.js alapú REST API, amely az adatokat szolgáltatja a kliens számára.
3. Adatbázis – Egy MySQL adatbázis, amely a számítógép-alkatrészek adatait tárolja.

A cél az, hogy a felhasználók könnyen, gyorsan és pontosan összeállíthassák PC konfigurációjukat, és az alkalmazás figyelmeztessen az esetleges kompatibilitási problémákra.

2.2. Funkcionális követelmények

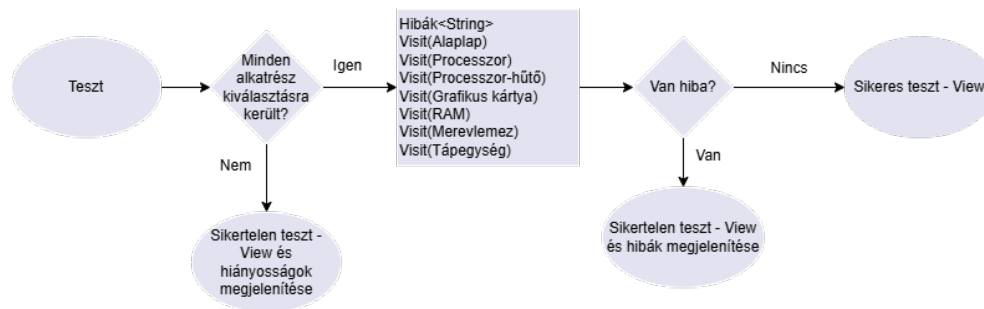
A célközönség, akik várhatóan ezt a programot hasznosnak találják majd, olyan személyek akik nem érdeklődnek az információs technológiák iránt és nem is szeretnének elmélyülni benne. Általában ezek az emberek nem tudnak számítógépet alkatrészerőlk alkatrészre megépíteni és a bonyolultabb kezelő felület is csak problémákat szülhet. Ezért a kezelő felület is a lehető legegyszerűbb kell hogy legyen.

Az alkalmazásnak a következő funkciókat kell biztosítania:

- Alkatrészválasztás: A felhasználók kiválaszthatják a különböző komponenseket (processzor, alaplap, RAM, grafikus kártya, tápegység, háttértár, hűtés).

- Kompatibilitás-ellenőrzés: Az alkalmazás ellenőrzi az egyes alkatrészek kompatibilitását.
- Hibaüzenetek megjelenítése: Ha egy vagy több alkatrész nem kompatibilis, az alkalmazás részletes visszajelzést ad a problémáról, illetve arról is ha még nem választottunk ki mindent.
- Felhasználóbarát felület: Könnyen kezelhető grafikus kezelőfelület, amely támogatja az egyszerű konfiguráció-összeállítást.

A Teszt funkció folyamatábrája:



2.1. ábra. Jól látható, hogy a tesztelés a Visitor Tervezési Mintával valósul meg.

2.3. Nem funkcionális követelmények

Az alkalmazásnak az alábbi nem funkcionális követelményeknek is meg kell felelnie:

Teljesítmény: A kompatibilitás-ellenőrzés gyorsan fusson (maximum egy másodperc alatt). Ezen felül a program a lehető legkevesebb erőforrást kell hogy használja, de kellően pontos számításokat kell hogy végezzen. A szervert is óvnia kell a leterheléstől, ezért minimálisra kell szorítani a vele történő kommunikációt is.

Skálázhatóság: Az alkalmazás minden egyes rétege késznek kell állnia egy későbbi könnyű és gyors, mérettől független bővítésre is. Beleértve az adatbázist, szervert, kezelőfelületet és magát a klienset is.

Felhasználóbarát kezelés: Az alkalmazás intuitív és könnyen kezelhető legyen. Minden funkció, gomb és visszajelzés legyen egyértelmű, pontos és lényegretörő.

Biztonság: Az adatbázis védelme biztosított legyen az illetéktelen hozzáférés ellen.

2.4. Adatmodell és adattárolás

Az adatokat MySQL adatbázis tárolja, amelyet a PHPMyAdmin kezel és minden részletének kötelezően eleget kell tennie a harmadik normálformának kivétel nélkül. Emel-

lett a relációk által létrejött környezetnek biztosítani kell a további bővítés (például új táblák létrehozása és relációk felépítése) elvégzésének egyszerűségét és gyorsaságát.

Az adatbázis főbb táblái: CPU (id, sorozat, modell, foglalat, fogyasztás stb.), CPU Hűtő (id, modell, zajszint, kompatibilitás), Alaplap (id, modell, foglalat, memória típusa stb.), RAM (id, modell, kapacitás, sebesség stb.), Grafikus kártya (id, modell, VRAM méret, fogyasztás stb.), Tápegység (id, modell, teljesítmény stb.), Merevlemez (id, modell, kapacitás, típus, interfész, stb.).

Ugyanúgy a kliens oldalon is tárolni kell az előre kiválasztott elemet vagy elemeket. Itt az újrahazsnosíthatóságra, skálázhatóságra és az elkülönítésre kerül a hangsúly.

2.5. Tervezési minták

Az alkalmazás fejlesztése során kiemelten fontos, hogy olyan tervezési mintákat alkalmazzak, amelyek elősegítik a kód tisztaságát, átláthatóságát, valamint könnyen karbantartható és bővíthető struktúrát biztosítanak [7]. Ennek érdekében az egyik legfontosabb architektúrális minta, amelyet alkalmaz a program, az az MVVM (Model-View-ViewModel [8]) volt, amely alapvetően meghatározta az alkalmazás szerkezetét és működését. Emellett a rendszer funkcionalitásának biztosítása érdekében a Visitor tervezési mintát is implementáltam, amely az alkatrészek kompatibilitásának ellenőrzésében játszott kulcsszerepet.

Alternatívaként választhattam volna az MVP (Model-View-Presenter [9]) vagy az MVC (Model-View-Controller [9]) mintát. Az MVP elsősorban webes alkalmazásoknál népszerű, míg az MVC szintén jó választás lehetett volna, de a WPF-nél az MVVM natív támogatást élvez, így sokkal praktikusabb megoldás.

2.5.1. MVVM (Model-View-ViewModel) tervezési minta

Az alkalmazás fejlesztésének alapjául az MVVM (Model-View-ViewModel) mintát szolgál, amely egy széles körben elterjedt architektúrális minta a WPF (Windows Presentation Foundation) alapú alkalmazások fejlesztésében. Az MVVM minta egyik legnagyobb előnye, hogy elkülöníti az adatok kezelését a felhasználói felülettől [8], ezáltal megkönnyíti a kód karbantartását és bővíthetőségét. Az MVVM három fő komponensből áll:

1. **Model (Adatmodell):** A Model réteg felelős az adatok tárolásáért. Az alkalmazásban az alkatrészek, például alaplapok, processzorok, memória modulok és egyéb hardverelemek adatai mind egy-egy Model-ben tárolódnak.
2. **ViewModel:** A ViewModel réteg biztosítja az adatok és a felhasználói felület közötti kapcsolatot. Felelős az üzleti logika végrehajtásáért, valamint az adatok

előkészítéséért és megjelenítéséért a View számára. Ez a réteg biztosítja az adatbázissal való kommunikációt, és egyértelmű interfészeket biztosít a View számára. A ViewModel kezeli az adatmódosításokat is és garantálja a megfelelő adatkapcsolatokat a felhasználói felületen. Az MVVM egyik legfontosabb eleme az adatköthetőség (data binding), amely lehetővé teszi, hogy a ViewModel automatikusan frissítse a View elemeit anélkül, hogy közvetlenül manipulálná azokat.

3. **View (Felhasználói felület):** A View réteg felelős az adatok vizuális megjelenítéséért és a felhasználói interakciók kezeléséért. Ebben a rétegben WPF alapú grafikus felhasználói felületet (GUI) kerül alkalmazásra, amely az MVVM struktúrának köszönhetően elkülönül az üzleti logikától és az adatkezeléstől.

Az MVVM minta alkalmazása jelentősen hozzájárul az alkalmazás átláthatóságához és skálázhatóságához. A különálló rétegek biztosítják, hogy az alkalmazás egyes részeit külön is módosítani lehessen anélkül, hogy az egész rendszert újra kellene írni. Például a felhasználói felület változtatása esetén a Model és a ViewModel rétegek érintetlenek maradnak, így a fejlesztési folyamat gyorsabb és hatékonyabb.

2.5.2. Visitor tervezési minta az alkatrészek kompatibilitásának ellenőrzésére

A másik jelentős tervezési minta, amely az alkalmazásba kerül, az a Visitor (Látogató) minta[7]. A Visitor minta egy viselkedési tervezési minta, amely lehetővé teszi, hogy egy objektumstruktúrában új műveleteket hajtsunk végre anélkül, hogy az adott objektumokat módosítanunk kellene.

Az alkalmazás esetében ezt a mintát az alkatrészek kompatibilitásának ellenőrzésére vesszük igénybe. Mivel a rendszerben különböző típusú hardverelemeket lehet kiválasztani és kombinálni (például alaplap, processzor, memória, videokártya stb.), szükséges egy olyan mechanizmus, amely ellenőrzi, hogy az egyes komponensek valóban kompatibilisek-e egymással.

A Visitor minta lehetővé teszi, hogy a kompatibilitás ellenőrzése az egyes hardverelemek módosítása nélkül hajtsódjon végre. Az alábbi módon kerül beépítésre:

Visitor interfész (Látogató osztály): Adott egy Visitor interfész, amely definiálja azokat a műveleteket, amelyeket az egyes hardverelemekkel végre lehet hajtani. Ez biztosítja, hogy minden alkatrész azonos interfészen keresztül legyen ellenőrizhető.

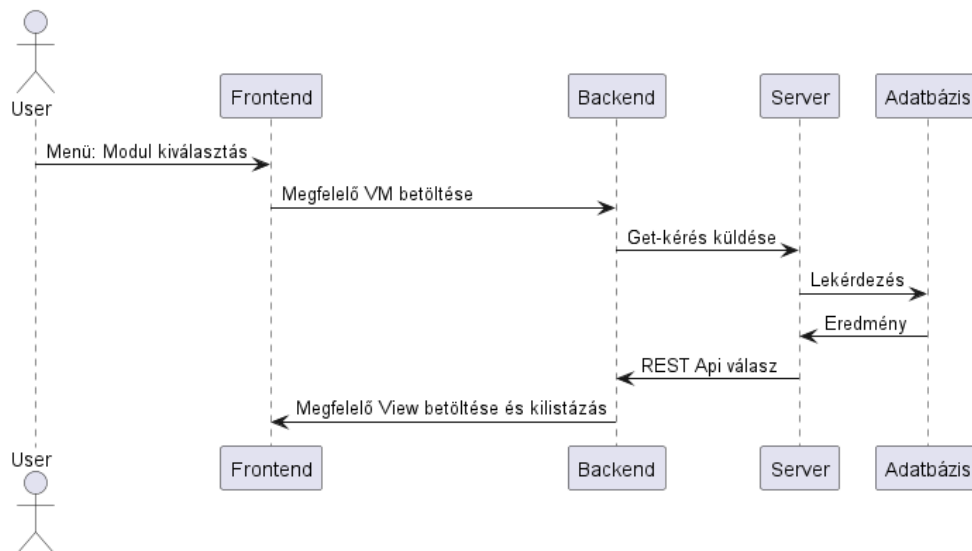
Hardverelemek mint „Accept” metódust implementáló osztályok: Minden egyes hardvertípus (pl. processzor, alaplap, memória) implementálja az Accept metódust, amely lehetővé teszi, hogy egy Visitor objektumot fogadjon [9]. Ez biztosította, hogy az egyes alkatrészek képesek legyenek meghívni a Visitor megfelelő metódusát, amely elvégzi az adott kompatibilitásellenőrzést.

Kompatibilitásellenőrző Visitor: A Visitor konkrét implementációja felelős az alkatrészek kompatibilitásának vizsgálatáért. Például ellenőrizte, hogy a választott processzor foglalata megfelel-e az alaplap foglalatának, vagy hogy a memória típusa kompatibilis-e az adott alaplappal.

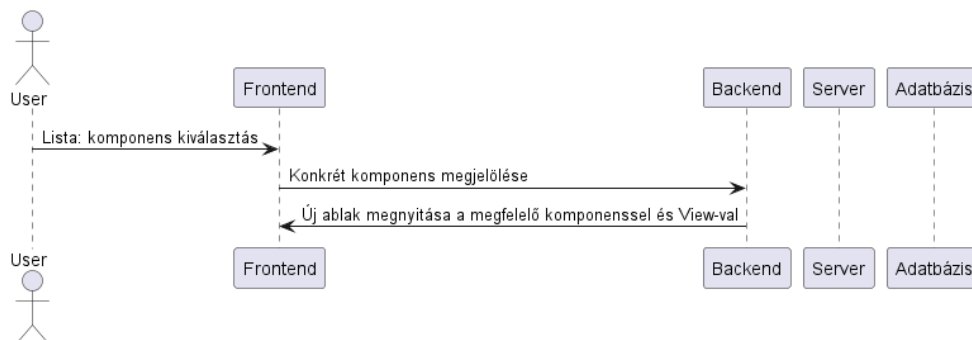
A Visitor minta alkalmazásának előnye az, hogy a kompatibilitásellenőrzés logikája teljes mértékben elkülönül az egyes hardverelemek osztályaitól. Ez azt jelenti, hogy ha a kompatibilitási szabályok változnak (például új alkatrészek jelennek meg a programban), akkor elegendő a Visitor osztályt módosítani anélkül, hogy az összes hardverosztályt át kellett volna írni.

2.6. Használati esetek (Use-case-ek)

A kiválasztott alkatrész-típus kilitázása:

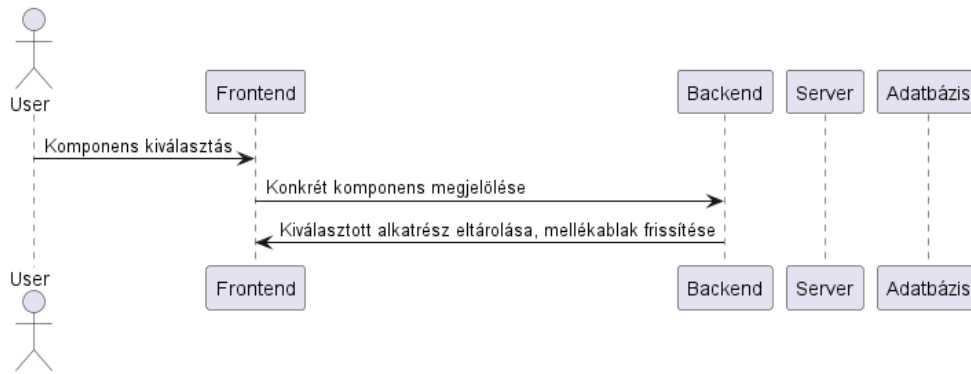


A kívánt modell további, terjedelmesebb adatainak megjelenítése:



2.2. ábra. Bővebb információ megjelenítés esetén, csak a backend fog dolgozni.

A kívánt modell kiválasztása:



2.3. ábra. Kiválasztáskor is minden folyamat a backend keretein belül fog zajlani.

2.7. Verziókezelés

A projekt fejlesztése során a GitHub [11] verziókezelő rendszert használok, amely egy elosztott verziókezelési megoldás, lehetővé téve a fejlesztések hatékony nyomon követését és az esetleges későbbi csapatmunkát. Maga a kód is itt tárolódik.

Három jól elkülönített ágon fut a fejlesztés. A master-ba kerül majd a végleges verzió, illetve ebből ágazódnak le az új ágak az elején (Server-Develop és Client-Develop). Server-Develop ágban a szerver fejlesztése zajlik. A Client-Develop branch-ben a kliens fejlesztése történik.

2.8. Korlátozások és jövőbeli fejlesztési lehetőségek

Az alkalmazás kizárólag Windows operációs rendszeren fut, mivel WPF technológiát használtam, amely kifejezetten a Windows platformra optimalizált. Ennek következtében a szoftver nem érhető el más operációs rendszereken, például macOS-en vagy Linuxon, ami korlátozza a felhasználhatóságát.

Továbbá az adatbázis karbantartása jelenleg manuálisan történik, ami azt jelenti, hogy ha új alkatrészek kerülnek piacra, azokat kézzel kell hozzáadni az adatbázishoz. Ez nemcsak időigényes, hanem potenciálisan hibalehetőséget is hordoz magában, hiszen az emberi beavatkozás miatt előfordulhatnak elírások vagy hiányzó adatok.

A jövőbeli fejlesztések során célok egy olyan rendszer kiépítése, amely lehetővé teszi az automatikus adatbázis-frissítést a gyártók által biztosított API-k segítségével. Ezzel jelentősen csökkenthető lenne a manuális munkavégzés, és az adatbázis mindig naprakész információkat tartalmazna az elérhető alkatrészekről.

Rövidtávon terveim között szerepel egy frissítés, amely egy világos témával is ellátott verzió. Applikáción belül, futás közben fog majd állítható lenni.

Ezen kívül szerepel egy mobilalkalmazás verzió fejlesztése, hogy a felhasználók bármilyen eszközről hozzáférhessenek az alkalmazáshoz, ne csak asztali környezetben. Ez jelentős mértékben növelné a felhasználói élményt és az alkalmazás elérhetőségét.

3. fejezet

Fejlesztési folyamat és rendszerfelépítés

3.1. Fejlesztési stratégia

A fejlesztési folyamat megkezdése előtt alaposan átgondoltam a projekt felépítését, valamint azt, hogy milyen sorrendben érdemes haladnom a megvalósítással. A célom az volt, hogy egy logikus, jól szervezett fejlesztési ütemtervet alakítsak ki, amely biztosítja a hatékony munkavégzést és a lehető legkevesebb módosítási igényt a későbbi szakaszokban.

A fejlesztést az adatbázis megtervezésével és kialakításával kezdtem. Ennek több indoka is volt. Az egyik legfontosabb tényező az volt, hogy az adatbázis felépítése határozza meg a teljes alkalmazás adatkezelési struktúráját, és így közvetlen hatással van mind a szerver, mind pedig a kliensoldali fejlesztésre. Az adatmodell meghatározása után pontosan ismertté vált, hogy milyen adatokat kell kezelni, milyen kapcsolatokat kell fenntartani az egyes entitások között.

Az adatbázis véglegesítése után következett a szerveroldali fejlesztés. Mivel a backend felelős az adatok kezeléséért és a klienssel való kommunikációért, kulcsfontosságú volt, hogy először ezt az architektúrát alakítsam ki. A szerveroldali implementáció során különös figyelmet fordítottam a hatékony adatkezelésre, az API kialakítására és a biztonsági mechanizmusokra. A cél egy stabil, gyors és skálázható szerver létrehozása volt, amely megfelelően kiszolgálja a kliensoldali alkalmazást.

Végül a kliensoldali fejlesztés következett. Ekkor már rendelkezésre állt egy működő adatbázis és egy REST API-t biztosító szerver rendszer, így a frontend fejlesztése során már egy konkrét szerveroldali struktúrához tudtam igazodni. A kliens fejlesztésekor az volt a legfontosabb szempont, hogy egy felhasználóbarát és intuitív felületet alakítsak ki, amely hatékonyan kommunikál a backenddel, és megfelelően kezeli a kapott adatokat.

Ez a fejlesztési stratégia biztosította, hogy minden réteg a korábban meghatározott és stabilan működő komponensekre épülhessen. A logikus sorrend követése minimálisra csökkentette a visszamenőleges módosítások szükségességét, ami jelentősen növelte a fejlesztés hatékonyságát és átláthatóságát.

3.2. Az adatbázis 1.0

Mint ahogy korábban már említettem végül a XAMPP program PHPMyAdmin kezelőfelületét választottam. Az adatbázis első verziójának fejlesztése jelentősen túllépte az általam előzetesen tervezett időkeretet, mivel a vártnál összetettebb rendszert kellett kialakítanom. Eredeti elképzelésem szerint az adatbázis körülbelül 15 táblából állt volna, amelyek közül hét az alapvető komponenseket (processzor, RAM, grafikus kártya, alaplap, processzorhűtő, tápegység és merevlemez) tárolta volna.

Azonban a kompatibilitási szempontok figyelembevételével az adatok jelentős részét közös struktúrába kellett szervezni. Az alkatrészek kompatibilitásának biztosítása miatt az adatbázis rekordjainak közel fele olyan attribútumokat tartalmazott, amelyek több táblában is előfordultak. Például az alaplap és a processzor esetében egyaránt szükség volt egy processzorfoglalat mezőre, amely kizárólag meghatározott értékeket vehetett fel. A redundancia elkerülése és az adatbázis strukturális tisztaságának megőrzése érdekében célszerű volt egy külön táblát létrehozni a processzorfoglalatok tárolására. Ez lehetővé tette, hogy az alaplap és a processzor táblája egyaránt hivatkozhasson erre a listára egy külső kulcs (foreign key) segítségével. Ez a megoldás hosszú távon előnyösebbé tette az adatkezelést és csökkentette az ismétlődő adatok tárolását, azonban a tervezési és implementációs fázisban jelentős többletmunkát és időráfordítást igényelt. Ezen logika mentén haladva alakítottam ki az adatbázis teljes szerkezetét, amely végül összesen harminhárom táblából állt. Ennek túlnyomó része valamilyen kiegészítő vagy segédtábla volt, amely az adatok normalizálását és a redundancia minimalizálását szolgálta. Bár az elsőre átgondolt és jól felépítettnek tűnő struktúra megfelelő alapot biztosított az alkalmazás számára, a fejlesztés előrehaladtával számos olyan problémába ütköztem, amelyek végül az adatbázis teljes újratervezését tették szükségessé.

3.3. Az adatbázis 1.0 hibái

Az egyik problémát az okozta, hogy az adatok nem kerültek megfelelően normalizálásra [13], így az adatbázis szerkezetében redundanciák és túlzott összetettségek jelentek meg. Ez különösen akkor válik problémássá, amikor az adatokat több különböző módon próbáljuk értelmezni és felhasználni, például a grafikus kártyák interfészét illetően.

Vegyük például a következő adatot: „PCIeX16 2x 16 Pin”. Ezt az adatot három különböző jellemzőre lehetne bontani:

1. Interfész típusa: Az adatátviteli csatlakozó típusa, amely itt a „PCIeX16”.
2. Pin csatlakozók száma és típusa: A csatlakozó, amely az áramellátásért felelős, és itt a „16 Pin” méretű csatlakozóra utal.
3. A szükséges csatlakozók száma: Az, hogy hány ilyen csatlakozóra van szükség a grafikus kártya megfelelő működéséhez, jelen esetben „2x 16 Pin”.

Ezek az információk valójában három különböző, egymástól független adatot képviselnek, amelyek egyetlen mezőben történő tárolása nemcsak logikai szempontból zavaró, hanem gyakorlati problémákhoz is vezethet. Ha ezt az adatot nem bontjuk le megfelelően, az később olyan nehézségeket okozhat a rendszer további kezelésében, hogy a kliensnek vagy a szervernek külön kell kezelnie az adatokat, miközben azok egyetlen mezőben vannak tárolva.

Ez a megoldás bár technikailag nem sérti a harmadik normálformát (3NF [14]), mivel az adat redundanciáját nem tartalmazza, mégis bonyolultabbá teszi a fejlesztést és a későbbi karbantartást. A kód komplexebbé válik, mivel több helyen kell az adatot külön-külön feldolgozni, ami lassítja a fejlesztési folyamatokat és növeli a hibalehetőségek számát. A cél mindig az, hogy az adatokat úgy strukturáljuk, hogy azok könnyen kezelhetők, világosan érthetők legyenek és minimalizáljuk a további karbantartásuk során fellépő bonyodalmakat.

A legjobb gyakorlat az, hogy az ilyen típusú adatokat megfelelően normalizáljuk és három különböző mezőként kezeljük őket. Így később egyszerűbben végezhetünk módosításokat, szűréseket és lekérdezéseket, miközben a kód és az adatbázis karbantartása is sokkal hatékonyabbá válik.

A másik óriási problémát az okozta, hogy mindenképpen már egy meglévő mezőt szerettem volna azonosítóként (primary key) használni. Hogy ezzel mi a probléma? Nagyon egyszerű szemléltetni. Vegyük például a Processzor tábla egy részletét.

Modell	Gyártó	Sorozat	Aljzat	Magok száma	Szálak száma	Gyorsaság
Varchar	Varchar	Int(FK)	Int(FK)	Int	Int	Int

A megfelelő azonosítás kérdése egy kulcsfontosságú szempont, és ebben esetben egyértelműen a modell lenne az ideális választás. A modell alapján történő azonosítás előnyös, mivel egy adott modellből csak egy létezik, és így biztosak lehetünk benne, hogy azonos típusú alkatrészekről beszélünk. Ezen belül pedig természetesen külön kezelhetjük azokat a variációkat, amelyek ugyanazon modellhez tartoznak, de kisebb különbségek vannak közöttük, például az egyik RGB megvilágítással, míg a másik nem. Az ilyen eltérések megfelelően kezelhetők, mivel az árazás a fő különbség, és az RGB jelző mező értéke egyszerűen „Igen” vagy „Nem” lehet. Ez a megoldás logikus és praktikus.

Azonban itt felmerül egy problémakör. Ha például a modell mezőt „Varchar(30)” típusra állítjuk, akkor problémák merülhetnek fel a keresés során. A modell alapján történő kereséskor ugyanis a rendszer minden egyes adatot átvizsgál, hogy megtalálja a keresett alkatrészt. Mivel a modellnevek gyakran hosszúak, például „MSI VENTUS 2X BLACK OC GeForce RTX 4060 8 GB Video Card”, ez rengeteg adat átvizsgálását eredményezi, ami jelentős teljesítménybeli problémákat okozhat, és hosszú keresési időt eredményezhet.

Emiatt, ha a modell mezőt „Varchar(30)” típusra állítjuk, az nem elegendő a hosszú nevek tárolására, és legalább 70 karakteres korlátot kellene beállítanunk, hogy minden lehetséges modellnevet megfelelően kezelni tudjunk. Ez azonban nemcsak helypazarló, hanem teljesítménybeli problémákat is okozhat, mivel így az adatbázis minden keresésénél hosszú karakterláncokat kell, hogy feldolgozzon, ami lassítja a rendszer működését [14].

Ahogy egyre mélyebben foglalkoztam ezzel a problémával, rájöttem, hogy a modellnév, amelyet eredetileg egy külön mezőben kívántam tárolni és megjeleníteni a kliensoldalon, valójában feleslegesen foglalná az adatbázis erőforrásait. Hiszen ezt az információt más módon is elő tudom állítani anélkül, hogy egy dedikált mezőt hoznék létre számára. De hogyan lehetséges ez?

Vegyük például a korábban említett „MSI VENTUS 2X BLACK OC GeForce RTX 4060 8 GB” elnevezésű videokártyát. Ha alaposan megvizsgáljuk ezt az elnevezést, láthatjuk, hogy valójában több kisebb, különálló információegységből áll:

1. MSI – a gyártó neve
2. VENTUS 2X BLACK OC – kiegészítő információ, amely megkülönbözteti a modellt a gyártó többi termékétől
3. GeForce – a márkanév (brand), amely a GPU családját jelöli
4. RTX – a technológiai jelölés, amely a kártya működési mechanizmusára utal
5. 4060 – maga a konkrét modell megnevezése
6. 8 GB – a VRAM (videomemória) mérete

Ha ezeket az adatokat már eleve külön tároljuk az adatbázisban megfelelő mezőkben (pl. Gyártó, Sorozat, Brand, Modell, VRAM stb.), akkor nincs szükség arra, hogy az egyes komponensekből képzett teljes modellnevet külön is tároljuk. Ehelyett a szerver vagy akár a kliensoldali alkalmazás is könnyedén össze tudja állítani ezt a megjelenítéshez, egyszerűen az egyes információk egymás mellé fűzésével.

Ez a megközelítés több szempontból is előnyös. Egyrészt csökkenti az adatbázis redundanciáját, hiszen nincs szükség egy hosszú, összetett karakterlánc külön tárolására, amely lényegében csak az egyébként is meglévő mezők adataiból épül fel. Másrészt javítja az adatbázis teljesítményét és optimalizálja a keresési folyamatokat, mivel a lekérdezéseknek nem kell egy hosszú szöveges mezőt átböngészniük az azonosítás során.

Emellett ez a módszer nagyobb rugalmasságot biztosít, hiszen ha később változik az adatmegjelenítés logikája – például más sorrendben kell megjeleníteni az információkat, vagy ki kell egészíteni további adatokkal –, ezt az alkalmazás szintjén könnyedén módosíthatjuk anélkül, hogy az adatbázis szerkezetén változtatni kellene.

Összességében tehát ezzel a módszerrel értékes erőforrásokat takaríthatunk meg, csökkenthetjük a rendszer komplexitását és egy hatékonyabb, skálázhatóbb adatkezelési struktúrát alakíthatunk ki.

Az adatbázis újratervezése mellett hozott végső döntésem nem csupán apróbb módosítások szükségességéből fakadt, hanem abból a felismerésből, hogy egy teljesen új, elméleti szinten kidolgozott, alapjaiban eltérő kapcsolatrendszerrel rendelkező adatbázis sokkal hatékonyabb és rugalmasabb lehetne a jövőbeli bővítések szempontjából. Egy ilyen struktúra lehetővé teszi a rendszer hosszú távú fenntartását és kezelhetőbb megoldásokat biztosít.

Az általam megtervezett új adatbázis egyik legnagyobb előnye, hogy könnyedén bővíthető anélkül, hogy a meglévő adatstruktúrát alapjaiban kellene módosítani. Egy ilyen kialakítás mellett, ha a fejlesztők a jövőben egy teljesen új alkatrészkategóriát kívánnak bevezetni, az adatbázis adaptációja minimális erőforrásigénnyel jár. Ahelyett, hogy egy teljesen új struktúrát kellene felépíteni, az új komponens egyszerűen integrálható a meglévő rendszerbe azzal, hogy felhasználjuk a már létező melléktáblákat és kapcsolatokat.

Például, ha egy új hardvertípus kerül bevezetésre, az adatbázis szerkezetében szinte csak egy új táblázatra van szükség, amely az adott alkatrész egyedi jellemzőit tárolja. Az összes többi, már létező és általános jellemzőket tartalmazó melléktábla – mint például csatlakozófej típusa, aljzat, gyártó, interfész, brand, mikroarchitektúra – újrahasználható. Ez a normalizált és moduláris felépítés jelentős előnyt biztosít a bővítések során, hiszen minimalizálja az új adattáblák számát, miközben megőrzi a rendszer koherenciáját és áttekinthetőségét.

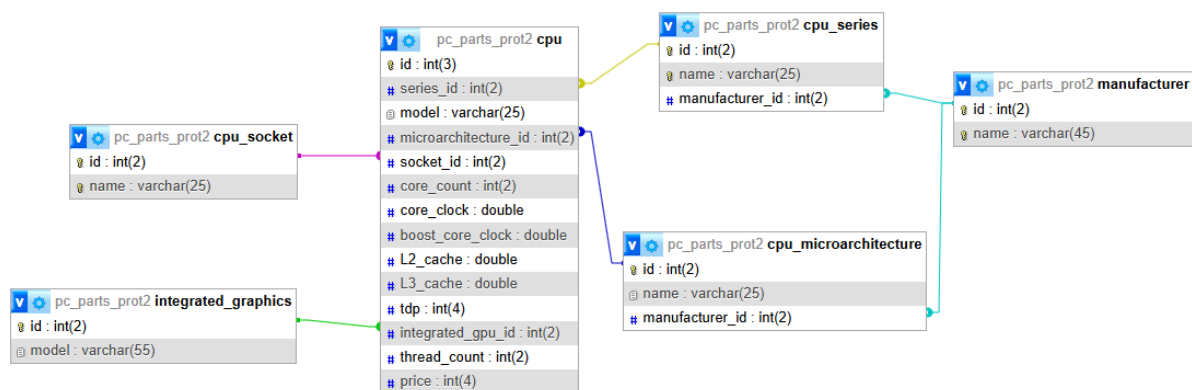
Az „1.0”-ás megközelítéssel ellentétben, ahol egy új hardvertípus bevezetése gyakran új adatstruktúrák kialakítását követelné meg, az újragondolt modellben az új komponens csak egy kis kiegészítést jelentene a meglévő rendszerben[14]. Ennek köszönhetően kevesebb hibalehetőséget biztosít.

3.4. Az adatbázis 2.0

Az előző szekcióban már ismertetésre került az a folyamat és az azokhoz kapcsolódó indokok, amelyek a második verziójú adatbázis megtervezéséhez és kialakításához vezettek. Ebben a fejezetben a rendszer technikai részleteit, valamint a mögöttes tervezési elveket és döntéseket részletezem, kiemelve az új struktúra előnyeit és azokat a megoldásokat, amelyek hatékonyabbá, átláthatóbbá és hosszú távon fenntarthatóbbá teszik az adatbázist.

Természetesen az adatbázis kialakítása során nem az volt a cél, hogy minden egyes komponens minden apró részletét eltároljam. A fókusz kizárólag azon alapvető információk rögzítésén volt, amelyek elengedhetetlenek annak meghatározásához, hogy a végső konfiguráció megfelelően működőképes lesz-e. Minden további részlet és extra adat kezelése a későbbi verziókban kap majd helyet, amikor az adatbázis funkcionalitását tovább bővítem. Minden egyes mezőnél arra törekedtem, hogy a lehető legszűkebb korlátokat adjam meg, miközben biztosítottam, hogy az adott adat még éppen elérjen benne.

3.4.1. A processzor és melléktáblái



3.1. ábra. Grafikusan jól látható a processzor és melléktábláinak kapcsolata.

CPU tábla

A cpu tábla a processzorok legfontosabb adatait tárolja és ez szolgál a CPU-k elsődleges adatforrásaként, és a következő mezőkből áll:

- **Id:** Egyedi azonosító, amely minden egyes CPU-t megkülönböztet.
- **series_id:** Egy idegen kulcs (FOREIGN KEY), amely a CPU sorozatát jelöli és a cpu_series táblára mutat. Ez segít abban, hogy az adott processzor melyik termékcsaládba tartozik.

- **model:** A CPU konkrét modellneve, amely lehet például „Ryzen 7 5800X” vagy „Intel Core i7-12700K”.
- **microarchitecture_id:** Egy másik idegen kulcs, amely a `cpu_microarchitecture` táblára mutat, és az adott CPU architektúráját [19] jelöli (például Zen 3, Alder Lake stb.).
- **socket_id:** Ez az oszlop az alaplap és a CPU közötti kompatibilitás biztosítására szolgál, és a `cpu_socket` tábla „id” mezőjére mutat.
- **core_count:** A CPU fizikai magjainak száma. Ez kulcsfontosságú információ a teljesítmény meghatározásában[18].
- **core_clock:** Az alap órajel (GHz-ben), amely megadja, hogy a CPU milyen sebességen működik normál körülmények között [19].
- **boost_core_clock:** Az a maximális órajel, amelyet a CPU turbó módban képes elérni, ha a terhelés megkívánja. Szintén GHz-ben.
- **L2_cache:** Az L2 gyorsítótár [19] mérete (MB), amely a CPU gyors adathozzáférését segíti elő.
- **L3_cache:** Az L3 gyorsítótár [19] mérete (MB), amely egy nagyobb, de valamivel lassabb gyorsítótár a CPU számára.
- **tdp:** A processzor hőtervezési teljesítménye (TDP – Thermal Design Power), amely megmutatja a maximális teljesítményfelvételt wattban, valamint azt, hogy mekkora hűtési kapacitás szükséges a megfelelő működéséhez.
- **integrated_graphics_id:** Egy idegen kulcs, amely az integrált grafikus vezérlőre vonatkozik (ha a CPU rendelkezik ilyennel) [19]. Ha a processzornak nincs integrált grafikus egysége, akkor ez a mező NULL értéket kaphat.
- **thread_count:** A processzor által támogatott szálak (threads) száma, amely az egyes magokon végrehajtható folyamatok számát mutatja [19]. Ez fontos tényező a többfeladatos munkavégzés és a teljesítmény szempontjából.
- **price:** A CPU ára, amely lehetővé teszi a felhasználók számára, hogy különböző modelleket ár alapján hasonlítsanak össze.

Szeretném ismét felhívni a figyelmet arra a megközelítésre, amelyet az Adatbázis 1.0 verziójánál már részletesen tárgyaltam. Az egyik legfontosabb tervezési elv az volt, hogy a termékek neve – amely végül a kliensoldali felületen kerül megjelenítésre – nem szükséges, hogy egy külön dedikált mezőben tárolásra kerüljön, hiszen az a meglévő adatokból dinamikusan előállítható.

Ahelyett, hogy egy hosszú karakterláncot rögzítenénk az adatbázisban minden egyes CPU-modell esetében, egyszerűen összefűzhetjük azokat a releváns információkat, amelyek az egyes processzorok azonosításához és megkülönböztetéséhez szükségesek. Ezek az adatok már egyébként is külön táblákban tárolódnak, így az alábbi elemek megfelelő sorrendbe való kombinálásával könnyedén létrehozható egy egyedi és jól strukturált név:

- Gyártó (Manufacturer) – Pl. „Intel” vagy „AMD”
- Sorozat (Series) – Pl. „Core i7” vagy „Ryzen 5”
- Modell (Model) – Pl. „12700K” vagy „5600X”
- Aljzat (Socket) – Pl. „LGA 1700” vagy „AM4”

Így például egy Intel Core i7 12700K LGA 1700 vagy egy AMD Ryzen 5 5600X AM4 elnevezés dinamikusan generálható anélkül, hogy ezt az információt redundánsan tárolnánk az adatbázisban.

Ezen kívül az ilyen strukturált megközelítés jelentősen leegyszerűsíti a rendszer karbantartását és bővíthetőségét. Ha például egy új processzorfoglat (socket) vagy új CPU-sorozat jelenik meg a piacon, akkor nem kell módosítani a meglévő táblázatok szerkezetét vagy adatkezelési logikáját. Mindössze annyit kell tenni, hogy az új foglatot vagy sorozatot a megfelelő melléktáblába (pl. `cpu_socket` vagy `cpu_series`) hozzáadjuk, és az új adatok azonnal integrálódnak a rendszerbe. Ez a megoldás nemcsak rugalmas és skálázható, hanem hosszú távon is fenntarthatóbbá teszi az adatbázist.

A melléktáblák szerepe és fontossága

A melléktáblák az adatbázis legegyszerűbb, mégis kulcsfontosságú elemei közé tartoznak. Elsődleges céljuk, hogy az adatok strukturált és könnyen kezelhető formában kerüljenek tárolásra [12].

Ezek a táblák általában egyedi azonosítóból és maximum két mezőből állnak, amelyeket a fő táblák – például a „cpu” tábla – a megfelelő kapcsolatokon keresztül használnak fel. Ennek az egyszerű felépítésnek köszönhetően az adatbázis könnyen bővíthető, frissíthető, valamint hatékonyabb lekérdezéseket tesz lehetővé.

Bár első ránézésre ezek a melléktáblák mellékesnek tűnhetnek, valójában kritikus szerepet játszanak a rendszer integritásának és fenntarthatóságának megőrzésében. A normalizált adatbázis-struktúrának köszönhetően, ha például egy új processzorsorozat vagy új foglat (socket) jelenik meg, akkor csupán a megfelelő melléktáblába kell beilleszteni az új adatokat anélkül, hogy az egész adatbázist módosítani kellene. Az ilyen melléktáblák tehát nem csupán egyszerű adatstruktúrák, hanem az adatbázis stabilitásának és rugalmasságának alapkövei.

Érdekes és fontos tervezési döntés, hogy a gyártó információja nem csupán a manufacturer táblában, hanem külön a `cpu_series` és a `cpu_microarchitecture` táblákban is megtalálható. Első ránézésre ez akár redundánsnak is tűnhet, de valójában jelentős előnyöket biztosít az adatbázis működése és a rendszer funkcionalitása szempontjából [13].

Miért van szükség a gyártó mezőre mindkét táblában? A fő indok az, hogy ez a megoldás lehetővé teszi egy hatékony és dinamikusan változó szűrőrendszer kialakítását a kliens- és szerveroldalon. Egy gyakori felhasználási esetet nézve:

A felhasználó kiválasztja a szűrőben a gyártót (pl. AMD vagy Intel). A rendszer ennek megfelelően módosítja a további elérhető szűrési opciókat, például csak az adott gyártóhoz tartozó processzorsorozatokat (`cpu_series`) és mikroarchitektúrákat (`cpu_microarchitecture`) jeleníti meg. Például ha a felhasználó AMD-t választja ki gyártónak, akkor az Intelhez tartozó mikroarchitektúrák és sorozatok automatikusan kiszűrésre kerülnek, hiszen AMD nem gyártja például a Lunar Lake architektúrát, és Intel sem gyárt Zen 2 architektúrát.

Miért nem elég csak a manufacturer táblát használni? Alternatív megoldásként a manufacturer táblán keresztül is lehetne az egyes CPU-khoz kapcsolni a gyártót, azonban így minden egyes lekérdezéshez több táblát kellene összekapcsolni (JOIN műveletekkel [12]), ami lassítaná a keresési folyamatokat és bonyolítaná a lekérdezési struktúrát.

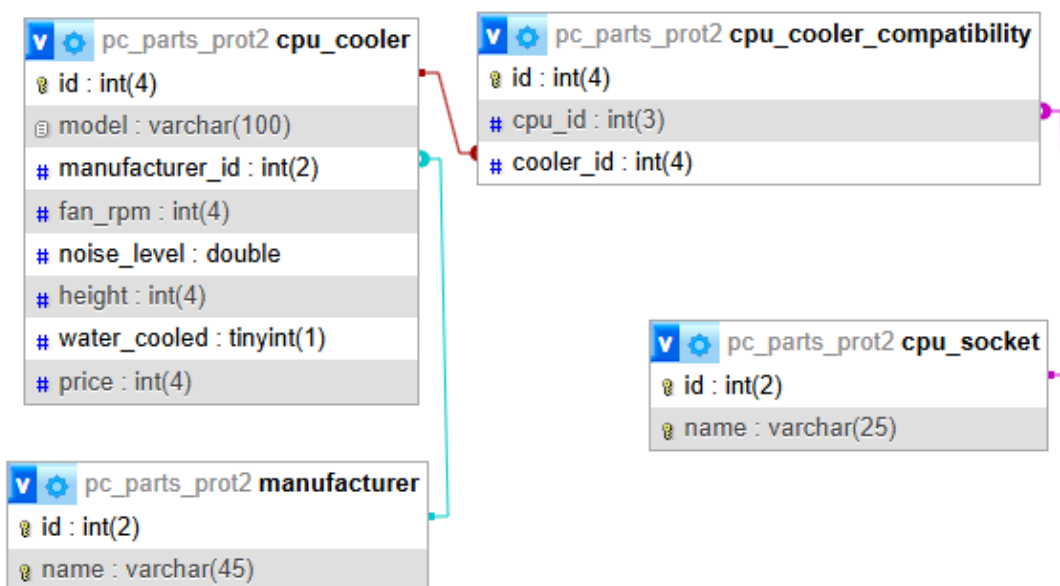
A gyártó információjának közvetlen tárolása a `cpu_series` és `cpu_microarchitecture` táblákban tehát nem csupán logikus és praktikus, hanem jelentős teljesítménybeli előnyökkel is jár. Az így kialakított adatbázis jobban skálázható, gyorsabb lekérdezéseket tesz lehetővé, és támogatja a felhasználóbarát, dinamikusan változó szűrőrendszereket.

3.4.2. A processzor hűtő és melléktáblái

A processzor-hűtő tábla

Ez a tábla biztosítja az információt a különböző típusú léghűtéses és vízhűtéses megoldásokról, amelyeket a felhasználók a processzoraik hűtésére használhatnak. A tábla az alábbi mezőkből áll:

- **id:** Egyedi azonosító, amely minden hűtőt megkülönböztet a többitől.
- **model:** A hűtő konkrét modellneve, amely lehet például Noctua NH-D15, Corsair iCUE H150i vagy Cooler Master Hyper 212 Black Edition.
- **manufacturer_id:** Egy idegen kulcs (FOREIGN KEY), amely a hűtő gyártójára mutat, és a manufacturer táblával áll kapcsolatban. Ez lehetővé teszi, hogy a hűtőket könnyedén gyártók szerint csoportosítsuk és szűrjük.



3.2. ábra. Van átfedés a processzor és a processzor hűtő melléktáblái között.

- **fan_rpm:** A ventilátor maximális fordulatszáma percenkénti fordulatszámban (RPM – revolutions per minute). Ez egy fontos adat, mert a magasabb fordulatszám általában nagyobb hűtési teljesítményt jelent, de egyben hangosabb működéssel is járhat.
- **noise_level:** A hűtő zajszintje decibelben (dB) mérve. Egy alacsony zajszinttel rendelkező hűtő halkabb működést biztosít, míg a nagyobb zajszintű modellek erősebb hűtési teljesítményt kínálhatnak, de zajosabbak is lehetnek.
- **height:** A hűtő magassága milliméterben. Ez kritikus szempont egy számítógép-ház kompatibilitása szempontjából, mivel egy túl magas hűtő nem biztos, hogy befér egy kisebb házba.
- **water_cooler:** Egy logikai (BOOLEAN) típusú érték, amely azt jelzi, hogy a hűtő vízűtéses-e (TRUE) vagy léghűtéses (FALSE) [19]. Ez segíthet a keresések és a szűrési lehetőségek optimalizálásában.
- **price:** A hűtő ára pénzben megadva. Ez lehetővé teszi az ár szerinti szűrést és összehasonlítást, így a felhasználók az ár-érték arány alapján dönthetnek.

A processzor-hűtő kompatibilitás tábla

A `cpu_cooler_compatibility` tábla egy kapcsolótábla, amely meghatározza, hogy egy adott processzorhűtő mely CPU foglalatokkal (socket-ekkel) kompatibilis. Ez N:N (many-to-many [14]) kapcsolatot valósít meg, mivel egy hűtő többféle CPU foglalattal

is kompatibilis lehet, és egy adott CPU foglalat több különböző hűtőhöz is használható [19].

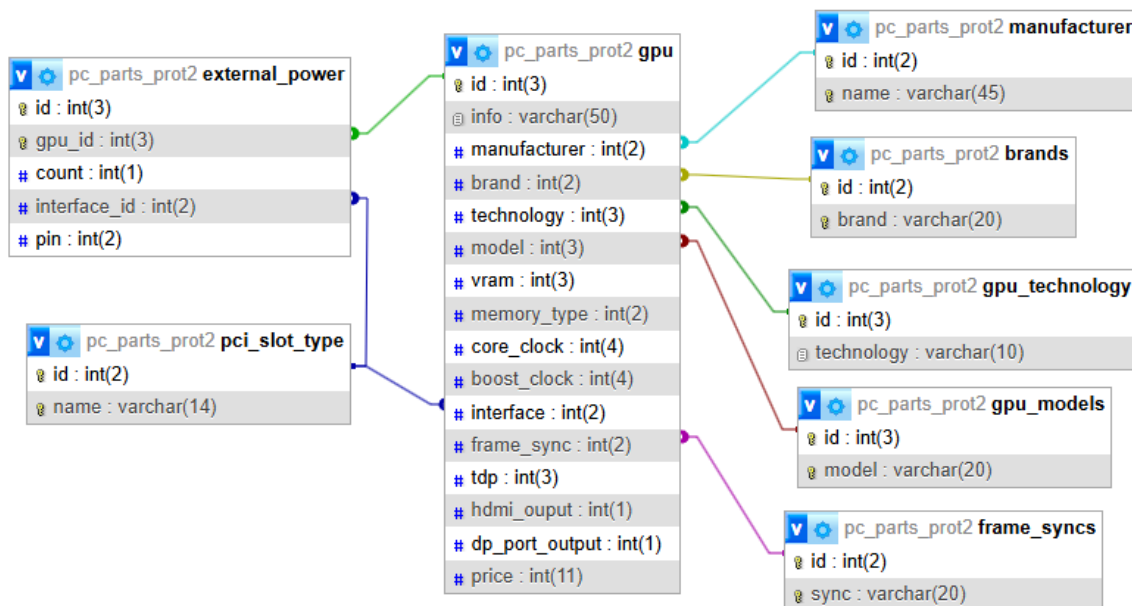
Ez a kapcsolat azért elengedhetetlen, mert nem minden hűtő támogat minden processzorfoglalatot – például egy AM4-es foglalatra tervezett léghűtő nem feltétlenül kompatibilis egy LGA1700-as Intel foglalattal. A `cpu_cooler_compatibility` tábla lehetővé teszi, hogy a rendszer egyszerűen és átláthatóan kezelje ezt a kompatibilitást, így a felhasználók gyorsan megtalálhatják a megfelelő hűtőt a CPU-jukhoz.

A tábla az alábbi mezőkből áll:

- **id**: Egyedi azonosító, amely minden kompatibilitási rekordot megkülönböztet.
- **cpu_socket_id**: Idegen kulcs (FOREIGN KEY) a `cpu_socket` táblából, amely meghatározza, hogy melyik processzorfoglalattal kompatibilis az adott hűtő.
- **cooler_id**: Idegen kulcs (FOREIGN KEY) a `cpu_cooler` táblából, amely azt jelöli, hogy melyik hűtőről van szó.

Mivel egy **N:N** kapcsolatot kezelünk, az adatbázis hatékony működéséhez ez a kapcsolótábla elengedhetetlen. Például egy Noctua NH-D15 kompatibilis lehet AM4, LGA1700 és TR4 foglalatokkal, míg egy Corsair iCUE H150i vízhűtéses rendszer csak AM4 és LGA1200 foglalatokhoz használható. Ezzel a megoldással az adatok tiszta, jól strukturált és könnyen bővíthető formában tárolhatók [12]. A többi melléktáblára már nem szeretnék kitérni, mert korábban már említésre került mind az aljzat, mind a gyártó tábla felépítése és szerepének fontossága is.

3.4.3. A grafikus kártya és melléktáblái



3.3. ábra. Az érdekesebb részlet a `pci_slot_type` tábla.

A grafikus kártya tábla

Szinte csak a legfontosabb tulajdonságait tárolja a grafikus kártyáknak, és ez az elsődleges adatforrás a GPU-kkal kapcsolatos információk kezelésére. A táblában található mezők lehetővé teszik a különböző modellek összehasonlítását és szűrését teljesítmény, memória, csatlakoztathatóság és energiaigény alapján.

A tábla mezői a következők:

- **id**: Egyedi azonosító, amely minden GPU-t megkülönböztet.
- **info**: Általános információ a GPU-ról, amely tartalmazhat rövid leírást.
- **manufacturer**: Idegen kulcs (FOREIGN KEY), amely a `manufacturer` táblára mutat, és jelzi a GPU gyártóját (például NVIDIA, AMD).
- **brand**: Idegen kulcs a `brands` táblából, amely a kártyát forgalmazó márkát jelöli (például ASUS, MSI, Gigabyte).
- **technology**: Idegen kulcs a `gpu_technology` táblából, amely a GPU működési architektúráját írja le (pl. RTX, GTX, RX) [18].
- **model**: Idegen kulcs a `gpu_models` táblából, amely a konkrét modell megnevezésére szolgál (pl. 4090, 7800 XT, 1080 Ti).

- **vram**: A GPU dedikált videomemóriájának [19] mérete (GB-ban).
- **memory_type**: A videomemória típusa [19] (pl. GDDR6, GDDR6X, HBM2).
- **core_clock**: Az alap órajel, amely megadja, hogy a GPU milyen sebességen működik normál körülmények között (MHz-ben) [19].
- **boost_clock**: A maximális órajel, amelyet a GPU turbó módban képes elérni (MHz-ben).
- **interface**: Idegen kulcs (FOREIGN KEY). A csatlakozási szabvány, amelyet a GPU használ az alaplaphoz való kapcsolódáshoz (pl. PCIe 4.0 x16) [19]. Az `external_power` tábla később részletezésre kerül.
- **frame_sync**: Idegen kulcs a `frame_syncs` táblából. Az adott GPU által támogatott képkockaszinkronizációs technológia (pl. G-Sync, FreeSync) [19].
- **tdp**: A grafikus kártya fogyasztása wattban megadva, amely fontos tényező a hűtési és tápegység-követelmények szempontjából.
- **hdmi_output**: Az elérhető HDMI csatlakozók száma a GPU-n.
- **dp_port_output**: Az elérhető DisplayPort csatlakozók száma a GPU-n.
- **price**: A GPU ára, amely segíti a felhasználókat az ár-érték arány szerinti döntésben.

A csatlakozó melléktábla

A `external_power` tábla a GPU-k külső tápcsatlakozóit kezeli, hiszen a modern grafikus kártyák teljesítménye gyakran meghaladja azt, amit az alaplap biztosítani tud. Ez N:1 kapcsolattal [12] kapcsolódik a `gpu` táblához, mivel egy GPU több tápcsatlakozóval is rendelkezhet.

- **id**: Egyedi azonosító.
- **gpu_id**: Idegen kulcs a `gpu` táblából, amely jelzi, hogy melyik GPU-hoz tartozik az adott tápcsatlakozó.
- **count**: A tápcsatlakozók száma. Egyes nagy teljesítményű GPU-k akár két vagy három külön tápcsatlakozót is igényelhetnek.
- **interface_id**: Idegen kulcs a tápcsatlakozó típusát tároló táblából (pl. PCIe, ATX 12VHPWR).
- **pin**: A csatlakozón lévő tűk száma (pl. 6-pin, 8-pin, 16-pin) [19].

Ezzel a megoldással hatékonyan követhető, hogy egy adott GPU milyen tápellátási követelményekkel rendelkezik, ami segíti a kompatibilitási ellenőrzést a tápegység kiválasztásakor. Érdeemes megemlíteni megint, hogy feldaraboltam apróbb adatokra egy hosszú, később nehezen kezelhető részt. „1x PCIeX16” mező értéket így később sokkal könnyebb lesz kezelni és aprólékosabban vizsgálni, illetve a módosításokra sem annyira érzékeny így már.

Az interface melléktábla

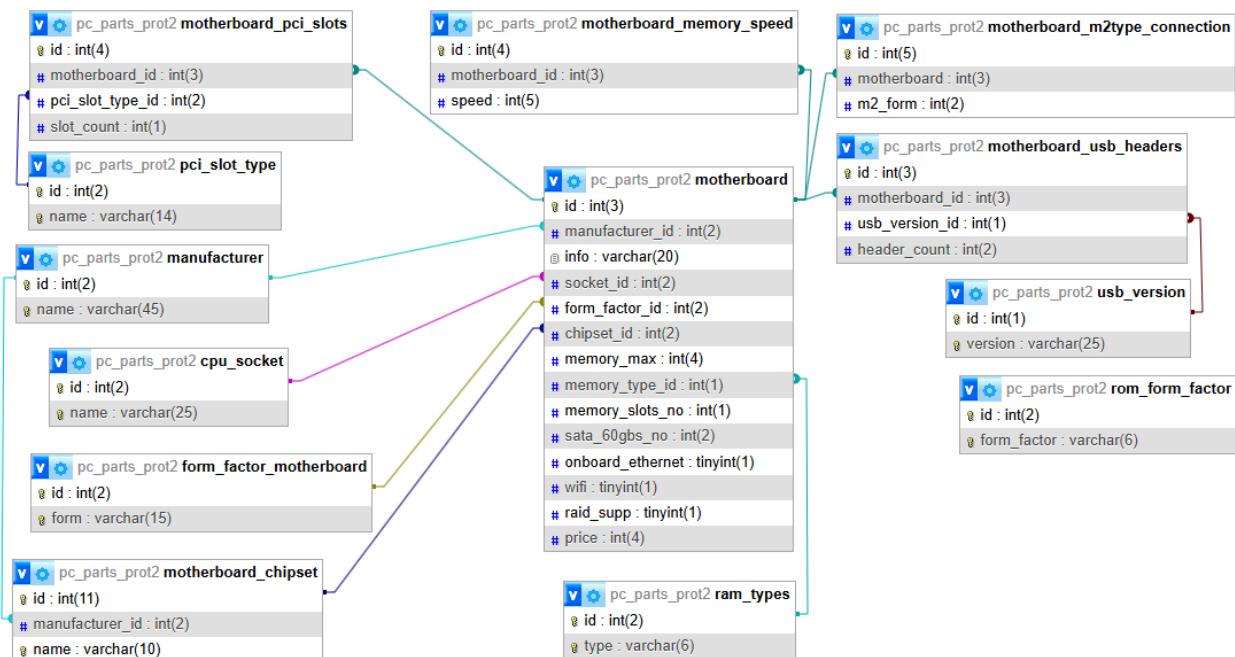
Ez az egyik legérdekesebb aspektusa a modularitásnak, hiszen az interface fogalma szinte minden komponensben visszaköszön, sőt, egy adott alkotóelemen belül is többször előfordulhat. A megfelelően kialakított interface tábla kulcsfontosságú szerepet játszik az adatbázis struktúrájában, mivel egyetlen közös entitásként szolgál több különböző funkcióhoz.

Például ez a tábla biztosítja az adatokat a GPU-k interfész mezőjéhez, amely meghatározza, hogy a grafikus kártya milyen PCIe csatlakozót használ az alaplaphoz való kapcsolódásra. Ugyanakkor ugyanez a tábla szolgál adatforrásként az `external_power` tábla `interface_id` mezőjének is, amely a grafikus kártya külső tápcsatlakozóit írja le.

Bár ebben az esetben két különböző típusú interfészről beszélünk (egyik az adatkapcsolatért, másik az energiaellátásért felelős), a funkciójuk alapvetően hasonló: egy meghatározott szabvány szerinti kapcsolat biztosítása két komponens között. Ahelyett, hogy két külön táblát hoznánk létre ezek kezelésére, egy egységes interfész rendszerrel oldjuk meg, amely sokkal rugalmasabb és könnyebben bővíthető.

Ezzel a megközelítéssel azonnal megóvtuk magunkat a felesleges új táblák és relációk létrehozásától, valamint időt és erőforrást takarítottunk meg az adatbázis kezelésében. Ha a jövőben egy új típusú interfész kerül bevezetésre, egyszerűen csak egy új rekordot kell hozzáadni az interfész táblához, anélkül, hogy az adatbázis struktúráját jelentősen módosítanánk. Pontosan ez az, amiért a modularitás kiemelten fontos: gyors alkalmazkodást tesz lehetővé a változó követelményekhez, miközben fenntartja az egyszerűséget és az átláthatóságot.

3.4.4. Az alaplap és melléktáblái



3.4. ábra. Nem kell megijedni, később szeparálva el lesz minden magyarázva.

Az alaplap tábla

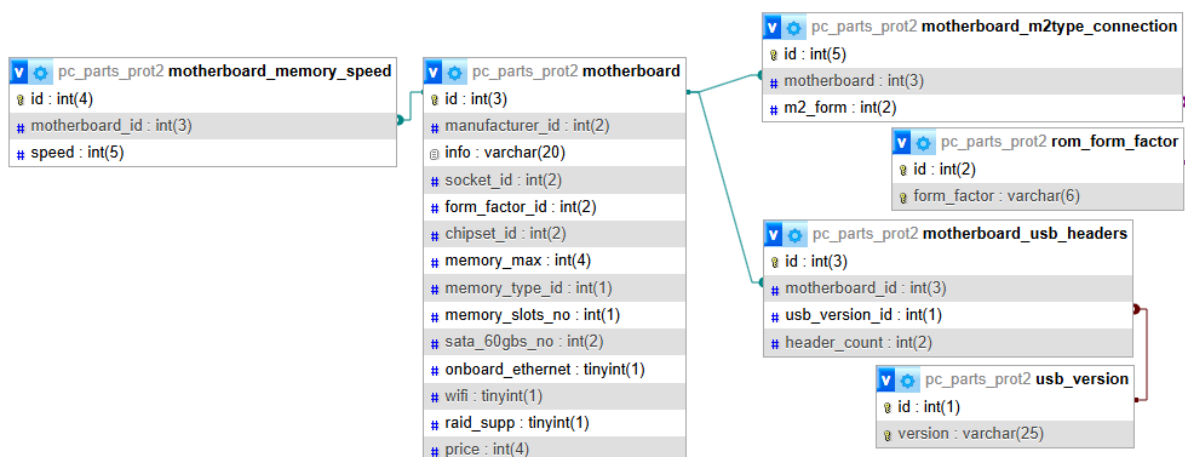
A motherboard tábla az alaplapok legfontosabb adatait tárolja, és az adatbázis egyik kulcsfontosságú eleme, hiszen minden más alkatrész ehhez csatlakozik [19]. Az alaplap meghatározza a kompatibilitást a különböző CPU-kkal, memóriákkal, háttértárolókkal és egyéb komponensekkel. A tábla az alábbi mezőkből áll:

- **id**: Egyedi azonosító, amely megkülönbözteti az egyes alaplaphoz tartozó rekordokat.
- **manufacturer_id**: Idegen kulcs, amely a gyártóra mutat a manufacturer táblából. Ez biztosítja, hogy könnyen szűrhetőek legyenek az egyes márkák termékei.
- **info**: Szöveges mező, amely további információkat tartalmazhat az alaplapról (például verziószám, különleges funkciók stb.).
- **socket_id**: Idegen kulcs a cpu_socket táblából. Meghatározza, hogy az adott alaplaphoz milyen processzorral rendelkezik, és így milyen CPU-k kompatibilisek vele.
- **form_factor_id**: Idegen kulcs a form_factor táblából. Az alaplaphoz milyen méretű és kialakítású alaplaphoz tartozik (például ATX, Micro-ATX, Mini-ITX) [19]. Ez befolyásolja, hogy az alaplaphoz milyen házba szerelhető be.

- **chipset_id**: Idegen kulcs a chipset táblából. Meghatározza az alaplap vezérlőchipjét, amely felelős a CPU, memória és egyéb komponensek kezeléséért. A chipset döntően befolyásolja az alaplap funkcionalitását és bővíthetőségét.
- **memory_max**: Az alaplap által támogatott maximális memória mennyiség (GB-ban megadva). Ez fontos tényező azok számára, akik nagy teljesítményű rendszert építenek.
- **memory_type_id**: Idegen kulcs a memory_type táblából. Meghatározza a támogatott memóriatípust (pl. DDR4, DDR5).
- **memory_slots_no**: Az alaplapon található memóriefoglalatok száma. Ez meghatározza, hogy hány RAM-modult lehet telepíteni az alaplagra.
- **sata_60gbs_no**: Az alaplapon található SATA 6.0 Gbps csatlakozók száma [19]. Ezek a csatlakozók a hagyományos HDD-k és SATA SSD-k csatlakoztatására szolgálnak.
- **onboard_ethernet**: Meghatározza, hogy az alaplap rendelkezik-e beépített Ethernet csatlakozóval [19].
- **wifi**: Boolean (Igen/Nem) érték, amely azt jelzi, hogy az alaplap rendelkezik-e beépített Wi-Fi modullal. Ez hasznos lehet azok számára, akik vezeték nélküli hálózati csatlakozást szeretnének használni.
- **raid_supp**: Boolean érték, amely azt mutatja, hogy az alaplap támogatja-e a RAID konfigurációkat a háttértárolókhoz [19]. A RAID különböző módjai (RAID 0, 1, 5 stb.) adatvédelemre és teljesítménynövelésre használhatók.
- **price**: Az alaplap ára, amely segíti a felhasználókat a különböző modellek ár szerinti összehasonlításában.

Ez a tábla is meglehetősen terjedelmes lett, azonban igyekeztem csak a legfontosabb adatokat megőrizni, elkerülve a felesleges információk tárolását. Úgy vélem, hogy önmagában az alaplap tábla szerkezete nem rejt különösebb érdekességeket vagy bonyolultabb megoldásokat. Éppen ezért érdemes inkább a hozzá kapcsolódó melléktáblákat és azok további összefüggéseit megvizsgálni, mivel ezek sokkal izgalmasabbak, és jól szemléltetik a modularitás előnyeit és gyakorlati alkalmazását.

3.4.5. 1:N kapcsolatú melléktáblák



3.5. ábra. Ebben a szekcióban kiderül, miért indokolt egy külön tábla a memória sebesség kezelésére.

A melléktáblák közül kezdjük a legeggyértelműbbekkel, például az M.2 foglalatokkal és az USB csatlakozókkal kialakított relációkkal.

M.2 foglalatok kezelése

Egy modern alaplapon gyakran több M.2 foglattal is rendelkezik, amelyek különböző méretű és sebességű meghajtókat támogatnak [19]. Egyes modellek esetében az egyes foglalatok fix mérettel rendelkeznek, míg más alaplapon állítható rögzítési pontokat kínálnak, lehetővé téve többféle M.2 SSD beszerelését ugyanabba a foglalatba. Például egy adott alaplapon támogathat 2242, 2260, 2280 és 22110 méretű M.2 SSD-ket egyetlen sloton belül. Ennek megfelelően a relációs adatbázisban célszerű az M.2 aljzatokat külön táblában tárolni, ahol minden alaplaphoz hozzárendelhetjük, hogy hány M.2 csatlakozóval rendelkezik, és ezek milyen méreteket támogatnak.

USB csatlakozók és azok sebessége

Az USB-csatlakozók esetében hasonló a helyzet: egy alaplapon jellemzően több USB port található, amelyek eltérő szabványokat és sebességeket képviselhetnek [19]. Egy adott alaplapon például rendelkezhet:

- Négy darab USB 3.1 Gen 1 (5 Gbps) csatlakozóval
- Kettő darab USB 3.2 Gen 2 (10 Gbps) csatlakozóval
- Egy USB-C porttal, amely akár 20 Gbps sebességre is képes

Ezeket az információkat nem célszerű az alaplap fő táblájában tárolni, hiszen minden modell eltérő portkiosztással rendelkezhet. Ehelyett egy külön USB-portokat kezelő táblában tároljuk az egyes alaplapokhoz tartozó csatlakozók számát és specifikációit. Például az én számítógépem is rendelkezik: Négy darab USB 3.1 és kettő darab USB 3.0 csatlakozóval, egy állítható M.2 foglalattal, amely négy különböző méretet (2242, 2260, 2280, 22110) képes fogadni.

RAM sebességek

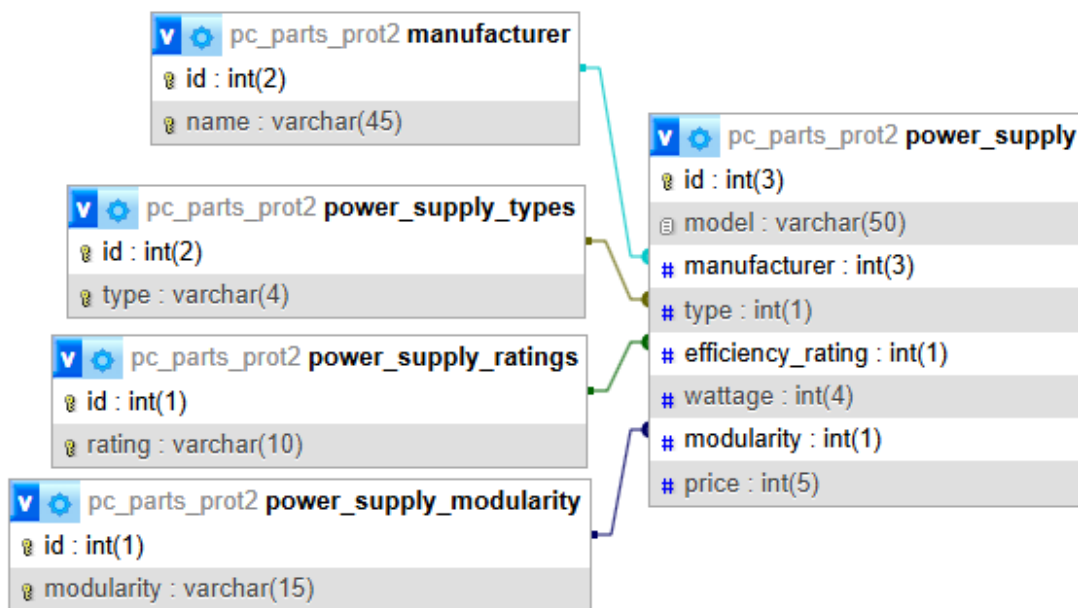
Bevallom, sokáig töprengtem a megfelelő megoldáson arra, hogyan érdemes ezt a problémát kezelni. Az első, kézenfekvő megoldás az lett volna, hogy egy hosszú VARCHAR mezőben tárolom vesszőkkel elválasztva a támogatott sebességeket. Ezt azonban hamar elvettem, mivel egy ilyen struktúra rendkívül nehézkesen kezelhető és szűrhető egy adatbázisban.

Felmerült bennem az a gondolat is, hogy egyáltalán nem szükséges eltárolni a memória sebességét, hiszen a legtöbb esetben, ha egy memória modul gyorsabb, mint amit az alaplap támogat, egyszerűen alacsonyabb órajelen fog működni, de nem okoz kompatibilitási problémát. Például, ha egy RAM 6000 MHz-es, de az alaplap csak 3200 MHz-et támogat, akkor a memória automatikusan 3200 MHz-re fog korlátozódni, anélkül hogy ez hibát eredményezne.

Mégis, végül úgy döntöttem, hogy létrehozok egy 1:N relációt és egy külön táblát a támogatott memória sebességek tárolására. Ennek fő oka az volt, hogy a gyártók a hivatalos weboldalaikon mindig külön hangsúlyozzák a kompatibilis sebességeket. Bár a legtöbb esetben egy memória sebesség visszaskálázása nem okoz gondot, előfordulhatnak olyan helyzetek, amikor egy alaplap hivatalosan nem támogat egy adott sebességet, és emiatt az adott RAM-modul egyszerűen nem működik megfelelően vagy egyáltalán nem indul el [19]. Egy konkrét példa erre, ha egy alaplap 6000 MHz-ig támogat memóriákat, de a kompatibilitási listáján 3000 MHz nem szerepel, csak 2800 MHz és 3200 MHz van feltüntetve. Ebben az esetben előfordulhat, hogy egy pontosan 3000 MHz-es RAM-modul nem lesz megfelelő, mert a BIOS és a memória kezelő algoritmusok nem rendelkeznek megfelelő profilokkal annak támogatására. Éppen ezért döntöttem a külön táblás megoldás mellett, amelyben egyértelműen tárolható, hogy az adott alaplap milyen sebességeket támogat hivatalosan, elkerülve ezzel az esetleges inkompatibilitási problémákat.

3.4.6. A tápegység és melléktáblái

A számítógép stabil működésének egyik kulcseleme a megfelelő tápegység kiválasztása, hiszen ez biztosítja a komponensek számára a megfelelő feszültséget és áramerősséget [19]. A tábla az alábbi mezőkből áll:

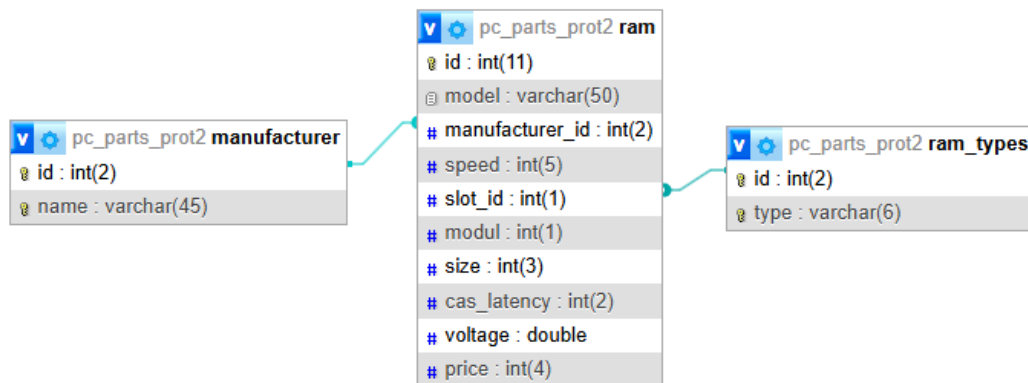


- **id:** Egyedi azonosító, amely minden tápegységet megkülönböztet.
- **model:** A tápegység modellneve, amely a konkrét terméket azonosítja (pl. RM750x vagy Focus GX-850).
- **manufacturer:** Idegen kulcs (FOREIGN KEY), amely a gyártót jelöli és a manufacturer táblára mutat. Ez lehetővé teszi, hogy egy gyártóhoz több különböző tápegység tartozzon.
- **type:** A tápegység fizikai mérete és formátuma, például ATX, SFX vagy TFX [19]. Ez azért fontos, mert az alaplapp és a számítógépház kompatibilitását biztosítja.
- **efficiency_rating:** Az energiahatékonysági besorolás, amely az adott tápegység hatékonyságát jelzi [19]. Tipikusan az 80 Plus minősítési rendszer kategóriáiba tartozik (pl. 80 Plus Bronze, Gold, Platinum, Titanium), és azt mutatja, hogy a bemeneti teljesítmény mekkora hányadát alakítja át hasznos energiává.
- **wattage:** A tápegység névleges teljesítménye wattban (W). Ez az érték meghatározza, hogy a tápegység mekkora terhelést képes kiszolgálni. Például egy 750W-os táp alkalmas lehet egy erősebb gaming PC-hez, míg egy 500W-os táp egy kevésbé erőforrásigényes konfigurációhoz megfelelőbb.
- **modularity:** Ez a mező azt jelzi, hogy a tápegység moduláris, félmoduláris vagy fix kábelezésű-e [19].
 - Moduláris: Az összes kábel leválasztható, így a felhasználó csak azokat a kábeleket használja, amelyekre szüksége van.

- Félmoduláris: Az alapvető kábelek (például a 24-pines alaplapi tápcsatlakozó és a CPU tápkábele) fixek, de a többi leválasztható.
 - Fix kábelezésű: Az összes kábel beépített, és nem lehet eltávolítani.
- **price:** A tápegység ára, amely lehetővé teszi az ár-érték arány összehasonlítását a különböző modellek között.

Miért fontos a power_supply tábla? A tápegység kiválasztása kulcsfontosságú a rendszer stabilitása és hosszú távú megbízhatósága szempontjából. Egy gyenge minőségű vagy alacsony hatékonyságú táp nemcsak a rendszer energiaellátását veszélyeztetheti, hanem akár hardverhibákhoz vagy károsodáshoz is vezethet [19]. A táblázat struktúrája lehetővé teszi a tápegységek gyors és pontos szűrését teljesítmény, hatékonysági besorolás vagy egyéb specifikációk alapján, ezáltal segítve a felhasználót a legmegfelelőbb tápegység kiválasztásában.

3.4.7. A RAM és melléktáblái

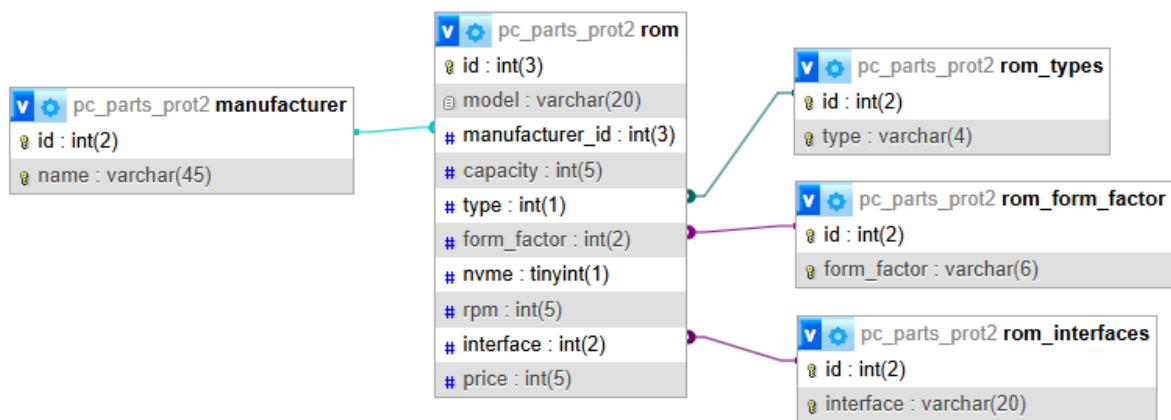


A RAM (memória) tábla a számítógép rendszermemóriájával kapcsolatos legfontosabb információkat tárolja. A memóriamodulok különböző sebességgel, kapacitással és csatlakozási szabványokkal rendelkeznek, így a megfelelő adatok struktúrált tárolása elengedhetetlen. Mezők és jelentésük:

- **id:** Egyedi azonosító, amely minden RAM modult megkülönböztet.
- **model:** A RAM modellt megnevező mező, amely például Vengeance LPX vagy Ripjaws V lehet.
- **manufacturer_id:** Idegen kulcs, amely a manufacturer táblára hivatkozik, ezzel jelezve, hogy melyik gyártó készítette a RAM-ot (pl. Corsair, Kingston, G.Skill, Crucial stb.).
- **speed:** A RAM sebessége (MHz-ben), amely megadja, hogy milyen órajelen képes működni [19]. Például 3200MHz, 3600MHz vagy akár 6000MHz.

- **slot_id**: Idegen kulcs, amely a RAM csatlakozási típusára (slot) vonatkozik [19]. Ez lehet DDR3, DDR4, DDR5, ami az alaplappal való kompatibilitás szempontjából kulcsfontosságú.
- **modul**: A RAM modulok száma egy adott készletben. Például 1x16GB, 2x8GB, 4x8GB, ami azt mutatja, hogy hány darab modulból áll a készlet.
- **size**: A RAM kapacitása (GB-ban). Ez lehet például 4GB, 8GB, 16GB, 32GB stb.
- **cas_latency**: A CAS késleltetés (CL) értéke, amely a memória késleltetését jelzi [19]. Például CL16, CL18 stb. Minél alacsonyabb az érték, annál gyorsabban képes reagálni a RAM.
- **voltage**: A RAM működéséhez szükséges feszültség (V-ban). Például 1.2V, 1.35V vagy 1.5V.
- **price**: A RAM ára.

3.4.8. A ROM és melléktáblái



A ROM (Read-Only Memory, ebben a kontextusban általános adattárolók – SSD-k, HDD-k, stb.) tábla az adatbázisban a számítógépes adattárolók főbb technikai paramétereit tartalmazza. Ebben a táblában a különféle típusú háttértárak – például SATA SSD-k, NVMe SSD-k, vagy hagyományos HDD-k – kerülnek rögzítésre. Mezők és jelentésük:

- **id**: Egyedi azonosító, amely megkülönbözteti az egyes ROM eszközöket.
- **model**: A meghajtó modellneve, például 980 PRO, WD Blue 1TB HDD, stb.

- **manufacturer_id**: Idegen kulcs, amely a manufacturer (gyártók) táblára mutat, jelezve, ki a meghajtó gyártója (pl. Samsung, Western Digital, Seagate, Kingston, Crucial stb.).
- **capacity**: A tárolókapacitás, jellemzően GB vagy TB mértékegységben megadva (pl. 256GB, 1TB, 2TB).
- **type**: A meghajtó típusa, például SSD, HDD, SSHD (hibrid meghajtó) [19]. Ez lényeges különbség, mivel más technológiákon alapulnak, és különböző teljesítményt nyújtanak.
- **form_factor**: A fizikai kialakítás, például 2.5", 3.5", M.2, U.2, amely megmutatja, milyen típusú foglalatba illeszkedik.
- **nvme**: Logikai mező (boolean vagy 0/1 típus), amely jelzi, hogy az adott tároló támogatja-e az NVMe protokollt (csak M.2 vagy U.2 SSD-k esetén értelmezhető) [19].
- **rpm**: A merevlemezek (HDD-k) forgási sebessége percenkénti fordulatszámban. Például 5400 RPM, 7200 RPM – az SSD-knél ez a mező értelemszerűen NULL érték lehet, hiszen nincs mozgó alkatrészük.
- **interface**: Az adatátviteli interfész típusa, például SATA III, PCIe Gen3 x4, PCIe Gen4, amely megmutatja, hogy milyen kapcsolaton keresztül kommunikál az alaplappal [19].
- **price**: Az adott meghajtó ára.

3.5. A szerver

A rendszer szíveként működő szerver felelős az adatbázis és a kliens közötti kommunikáció biztosításáért. Feladata, hogy a kliens által küldött kérésekre reagáljon, lekérdezze az adatokat az adatbázisba, majd visszaküldje a választ a felhasználónak. A szerver biztosítja az adatok épségét, a logikai összefüggések betartását, valamint a különféle szűrési és ellenőrzési folyamatokat.

3.5.1. Szerverindító fájl – server.js

```

1  const http = require('http');
2  const app = require('./app');
3  const port = process.env.port || 3000;
4  const server = http.createServer(app);
5  server.listen(port);

```

A `server.js` fájl a Node.js-alapú alkalmazás belépési pontja, vagyis innen indul el maga a szerver. Az `http` modul segítségével létrehoz egy egyszerű HTTP szerveret, amely a `./app` modulból (egy Express alkalmazás) származó logikát kezeli. A `server.listen(port)` sor elindítja a szerveret a megadott porton, ami alapértelmezetten a 3000, de környezeti változóval (pl. `process.env.port`) könnyen testre szabható éles környezetben is [1].

3.5.2. Csomagkezelés – `package.json` és `package-lock.json`

A `package.json` fájl az alkalmazás konfigurációs központja. Tartalmazza:

- az alkalmazás nevét, verzióját,
- a szükséges függőségeket (pl. `express`, `cors`, stb.),
- a futtatáshoz vagy fejlesztéshez szükséges scripteket (pl. `start`, `dev`),
- valamint egyéb metaadatokat.

A `package-lock.json` ezzel szemben részletesen, pontos verziókkal rögzíti az összes telepített csomagot és azok függőségeit. Ez garantálja, hogy a projekt minden környezetben pontosan ugyanazokat a csomagokat használja, elkerülve az esetleges inkompatibilitásokat vagy eltéréseket [1].

3.5.3. Az alkalmazás belső motorja – `app.js`

Az `app.js` fájl az alkalmazás szívéét képezi, itt történik az összes központi beállítás, amely lehetővé teszi az API működését. Ez a fájl a Node.js és az Express keretrendszer segítségével kezeli az útvonalakat (route-okat), beérkező kéréseket és válaszokat, valamint a middleware-ek alkalmazását [1, 2].

Express keretrendszer inicializálása

```
1 const express = require('express');  
2 const app = express();
```

Itt az `express` meghívásával létrejön az alkalmazás példány, amelyre az útvonalakat és middleware-eket beállíthatjuk [2].

```
1 const motherboardRoutes = require('./api/routes/motherboards');  
2 const cpu_coolerRoutes = require('./api/routes/cpu_coolers');  
3 ...
```

Ezek a sorok beemelik az egyes komponensekhez tartozó útvonalkezelő modulokat (route-ok), amelyeket később csatolunk az alkalmazáshoz. Ez strukturáltabbá, modulisabbá és könnyebben karbantarthatóvá teszi az alkalmazást [2].

3.5.4. Middleware-ek beállítása

```
1 const bodyParser = require('body-parser');
2 app.use(bodyParser.urlencoded({extended: false}));
3 app.use(bodyParser.json());
```

A body-parser modul lehetővé teszi, hogy az Express alkalmazás képes legyen értelmezni a beérkező HTTP-kérések törzsében (body) található adatokat – például JSON vagy URL-kódolt formában. Ez elengedhetetlen a POST vagy PUT kérések feldolgozásához [2].

3.5.5. Útvonalak (route-ok) regisztrálása

```
1 app.use('/motherboards', motherboardRoutes);
2 app.use('/cpu_coolers', cpu_coolerRoutes);
3 ...
4 module.exports = app;
```

Ezek a sorok hozzárendelik az importált útvonalkezelő modulokat az adott URL-előtaghoz. Például ha a kliens a /cpus végpontot hívja meg, akkor a ./api/routes/cpus.js fájl logikája fog lefutni [1]. Ez a szétválasztás tiszta és átlátható szerkezetet eredményez: minden fő komponens (CPU, GPU, RAM stb.) saját különálló útvonalkezelőt kap, így a kód olvashatóbb, a fejlesztés pedig sokkal könnyebben skálázható. Végül az app objektum exportálásra kerül, hogy azt más fájlok – például a server.js – importálhassák és elindíthassák a szerveret az alkalmazás logikájával.

3.5.6. Útvonalak kezelése - Példa: motherboards.js

Ez a fájl az Express keretrendszer Router objektumával készült, és az alaplappal kapcsolatos adatokat szolgáltatja a kliens számára [2]. Különlegessége, hogy nemcsak az alaplappok főbb jellemzőit adja vissza, hanem a hozzájuk tartozó bővítési lehetőségeket is: például az M.2 típusokat és az USB header konfigurációkat is.

Alapbeállítások

```
1 const express = require('express');
2 const router = express.Router();
3 const db = require('../db');
```

Az Express Router példánya lehetővé teszi, hogy modulárisan, elkülönítve kezeljük az útvonalakat. A db modul a MySQL adatbáziskapcsolatot tartalmazza, így az aszinkron lekérdezések biztosítva vannak [2].

```
router.get('/', async (req, res) => {
```

Ez az útvonal lekér minden szükséges adatot az alaplapokról és azok kapcsolódó tulajdonságairól. Egyszerre három SQL lekérdezést futtat le:

Alaplap fő adatainak lekérése

```
1  const [motherboards] = await db.query(`SELECT motherboard.id,
    ↳ manufacturer.name AS manufacturer,
2    cpu_socket.name AS socket, form_factor_motherboard.form AS
    ↳ form_factor,
3    motherboard_chipset.name AS chipset, motherboard.memory_max,
4    ram_types.type AS ram_type, motherboard.memory_slots_no,
5    motherboard.sata_60gbs_no, motherboard.onboard_ethernet,
6    motherboard.wifi, motherboard.raid_supp, motherboard.price
7  FROM motherboard
8  JOIN manufacturer ON motherboard.manufacturer_id = manufacturer.id
9  JOIN form_factor_motherboard ON motherboard.form_factor_id =
    ↳ form_factor_motherboard.id
10 JOIN cpu_socket ON motherboard.socket_id = cpu_socket.id
11 JOIN motherboard_chipset ON motherboard.chipset_id =
    ↳ motherboard_chipset.id
12 -- stb.`);
```

Ezek a csatolt mezők lehetővé teszik, hogy ne csak az idegen kulcsokat (ID-ket) kapjuk vissza, hanem azonnal emberi olvasható formában is megkapjuk az információkat.

Alaplap M. 2 támogatásainak lekérése

```
1  const [m2s] = await db.query(`
2  SELECT motherboard, rom_form_factor.form_factor
3  FROM motherboard_m2type_connection
4  JOIN rom_form_factor ON rom_form_factor.id = m2_form
5  `);
```

Ez az 1:N kapcsolat azt vizsgálja, hogy egy alaplap milyen típusú M.2 meghajtókat támogat. Egy alaplap többféle M.2 formátumot is fogadhat (pl. 2280, 22110), ezért ezt külön kapcsolótáblával tároljuk. Pontosan így lett megoldva az USB típusok tárolása is, mert egy alaplap többféle USB csatlakozóval rendelkezhet.

3.5.7. JSON válasz

```
1 res.status(200).json({
2   motherboards: motherboards,
3   m2types: m2s,
4   usb_headers: usb_headers
5 });
```

A válasz három tömbből áll:

- motherboards – fő információk minden alaplapról
- m2types – az egyes alaplaphoz tartozó M.2 formátumok
- usb_headers – az alaplapok USB fejléc konfigurációja

Ez lehetővé teszi, hogy a kliensoldali alkalmazás könnyen összekapcsolja az adatokat, és felhasználóbarát módon jelenítse meg őket.

3.5.8. Hibakezelés

```
1 catch (error) {
2   console.error("SQL␣Error:", error);
3   res.status(500).json({ error: error.message });
4 }
```

Összefoglalás

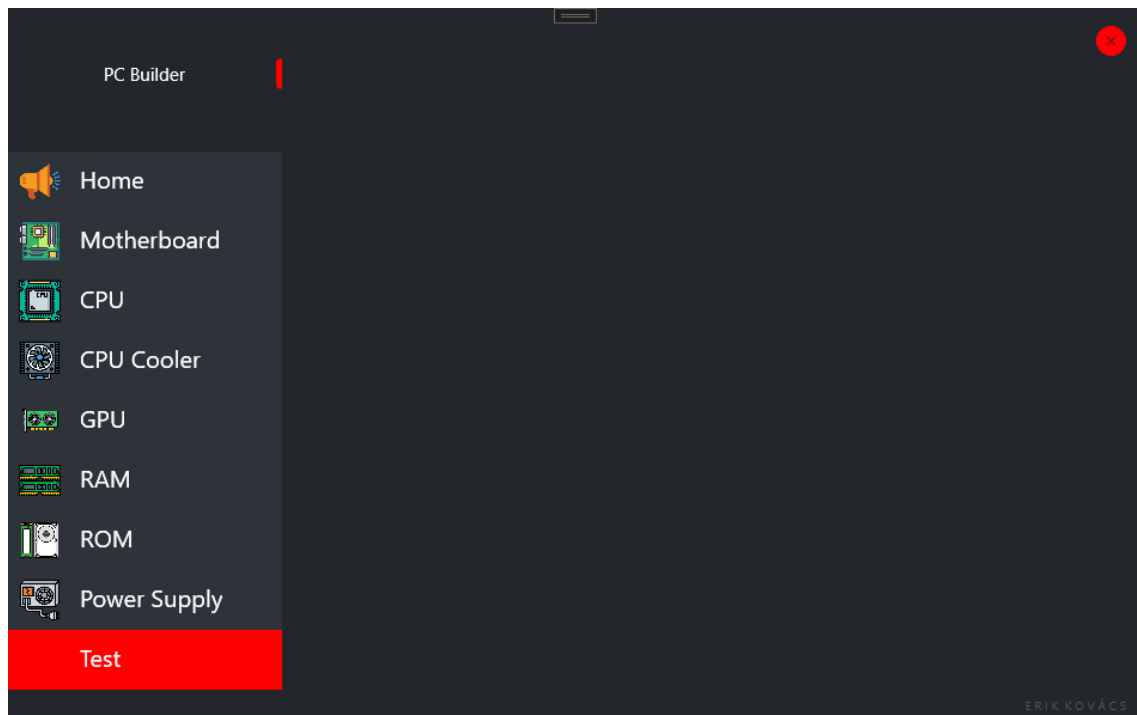
A motherboards.js egy strukturált route-fájl, amely komplex kapcsolatok kezelésére is képes, miközben megőrzi az egyszerűséget a kliens számára. A moduláris adatbontás lehetővé teszi a jövőbeni bővítést és új kapcsolatok hozzáadását is, miközben kiválóan demonstrálja az 1:N relációk gyakorlatban való hasznosságát. Egyértelmű példája annak, hogyan jelenik meg az adatbázis-normalizálás és a modularitás az API-k tervezésében [2].

3.6. A kliens

A kliens kezelőfelületeit igyekeztem a lehető legintuitívebben és logikus elrendezésben kialakítani, hogy még azok számára is könnyen kezelhető legyen, akik nem rendelkeznek mélyebb informatikai tudással. A design tekintetében a célom az volt, hogy letisztult és egyszerű vizuális élményt nyújtsak, amely segíti a felhasználókat a navigálásban és az információk gyors megértésében. Figyelembe vettem azt is, hogy a célközönség olyan felhasználók, akik nem hajlandóak hosszasan elmélyedni a technikai részletekben, és

akik nem töltenek sok időt számítógép előtt, például fórumok olvasásával. A felület tehát nemcsak esztétikailag kellemes, hanem a funkcionalitását tekintve is könnyen elérhető és gyorsan használható a felhasználók számára.

3.6.1. A főoldal



3.6. ábra. A főoldal kinézete.

Ahogy a mellékelt képen is jól látható, a kezelőfelület bal oldalán helyezkedik el egy menüoszlop, amely kilenc különböző opciót kínál a felhasználónak. A menüponthoz kattintva – a Teszt gomb kivételével – a tőlük jobbra található nagyobb, üres tartományban dinamikusan betöltődik a választott menüponthoz tartozó nézet (View) és a hozzá kapcsolódó nézetmodell (ViewModel). Ezt a tartalomkezelést egy Content-Controller végzi, amely gondoskodik arról, hogy mindig a megfelelő páros jelenjen meg. Fontos megemlíteni, hogy minden új menüpont kiválasztásakor ugyanabban az ablakterületen, ugyanott frissül a tartalom, lecserélve az előző nézetet az újra. Ez a megoldás nemcsak esztétikus és áttekinthető, hanem technikailag is hatékony, mivel valós idejű frissítést biztosít, ezzel is növelve a felhasználói élményt.

3.1. Kódrészlet. ContentControl ahol megjelenítem az adott View-t.

```
1 <ContentControl Grid.Row="1" Content="{Binding_SelectedViewModel}"  
   ↪Margin="0,0,0,0"/>
```

Az ablak alapvető szerkezeti felépítése egy Grid elrendezés, amely lehetővé tette számomra, hogy gyorsan és hatékonyan felosszam a felületet a szükséges szekciókra. A

bal oldalon egy vertikális orientációjú Menü komponenst alkalmaztam, amely egy külső fájlból tölti be a stílusra vonatkozó adatokat, hasonlóan ahhoz, ahogyan a HTML és CSS páros működik. Ez a megoldás segít a rugalmasabb és könnyebben átlátható felhasználói felület kialakításában. Érdekesség, hogy az ablakbezáró gomb formáját nem a stílusának módosításával értem el, hanem a gomb tartalmát egy piros .png kép fájl alkalmazásával. Ez a megoldás gyorsabb és hatékonyabb volt a fejlesztés szempontjából, miközben a kinézete szinte teljesen megegyezik azzal, mintha kódolással próbáltuk volna megvalósítani a gombot.

3.2. Kódrészlet. Itt csatolom a sablon egyik tulajdonság-csoportját a menü elemhez.

```
1 Template="{StaticResource␣Menu_Template}"
```

Szintén ebben az elkülönített sablon - csoportokat tároló .xaml fájlban van definiálva a gombok kinézete és szerkezeti felépítése.

3.3. Kódrészlet. Maga a menüben lévő elemek tulajdonsága és felépítésének a külső sablonjának fejléce.

```
1 <ControlTemplate x:Key="Menu_Template"
2     TargetType="{x:Type␣MenuItem}">
3     <Border x:Name="border"
4         Background="#2E333A"
5     ...
6 </ControlTemplate>
```

A ViewModel-ben egy teljes (full) property-t biztosítunk, amely lehetővé teszi a folyamatos állapotváltozást és adatfrissítést. Kiemelném az OnPropertyChanged() metódus szerepét: ez figyeli az adott property értékének változását, és automatikusan értesíti a felhasználói felületet (View-t), hogy frissítse a megjelenített adatokat. Ennek köszönhetően valós időben tud változni a megjelenés, amikor a háttérben módosul az adat [16].

3.4. Kódrészlet. Innen kapja a ContentControl a friss View-t.

```
1 public BaseViewModel SelectedViewModel
2 {
3     get { return selectedViewModel; }
4     set
5     {
6         selectedViewModel = value;
7         OnPropertyChanged(nameof(SelectedViewModel));
8     }
9 }
```

Mi okozza a property értékének változását?

Amikor a felhasználó bármelyik menüelemre kattint, egy Command aktiválódik, amely a kapott paraméterek alapján módosítja a selectedViewModel értékét [20].

3.5. Kódrészlet. A menü elemének nyitó része.

```
1
2 <MenuItem Header="Motherboard" Template="{StaticResource Menu_Template
  ↳}" Command="{Binding UpdateViewCommand}" CommandParameter="
  ↳Motherboard">
```

Jól látható, hogy a ViewModel-ben tárolt Command meghívásra kerül, és átadjuk neki a „Motherboard” paramétert. A Command-on belül egy Switch szerkezet segítségével a megfelelő ViewModel-t hozzuk létre, amely azután frissíti a selectedViewModel mező értékét a főoldal ViewModel-jében. Ezzel aktiválódik az OnPropertyChanged() függvény, ami értesíti a felhasználói felületet a változásról [8].

3.6. Kódrészlet. Itt dől el, hogy melyik View kerül megjelenítésre.

```
1 public void Execute(object parameter)
2 {
3
4     switch (parameter.ToString())
5     {
6         case "Home":
7             viewModel.SelectedViewModel = new HomeViewModel();
8             break;
9         case "Motherboard":
10            viewModel.SelectedViewModel = new MotherboardViewModel();
11            break;
12        case "CPU":
13            viewModel.SelectedViewModel = new CPUViewModel();
14            break;
15        ...
16        default:
17            break;
18    }
19 }
```

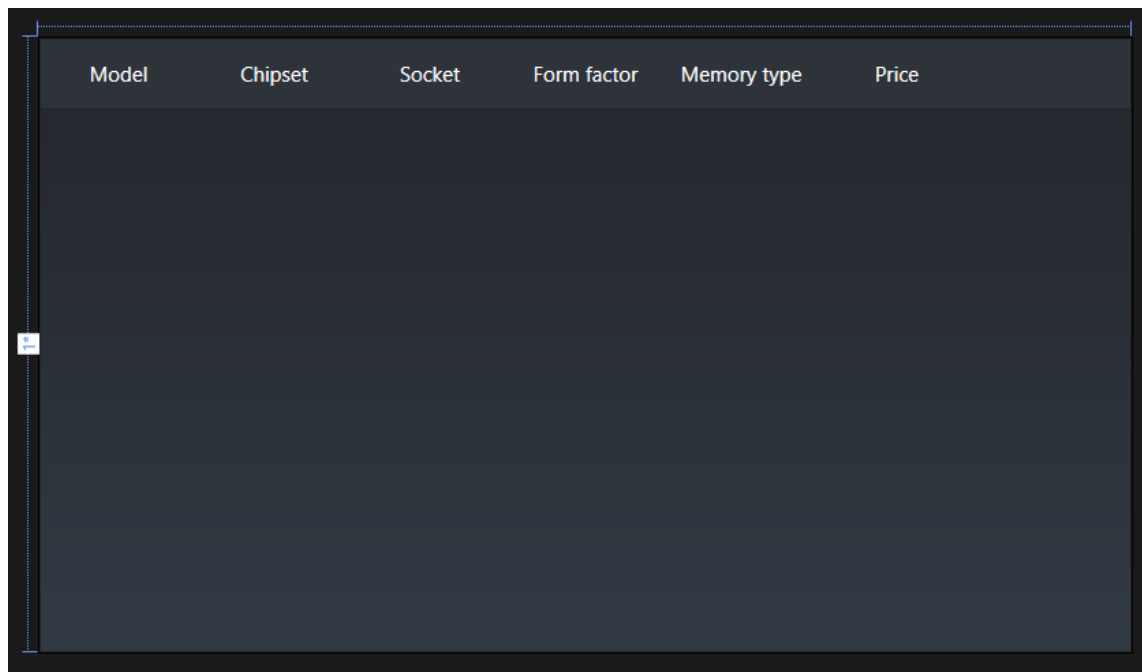
Azonban felmerülhet két kérdés is. Először is, ha jól láthatóan ViewModel-eket adunk vissza, miért változik akkor a View? A válasz egyszerű: mivel minden View és ViewModel valamilyen módon össze van kötve. Ha egy ViewModel létrejön, a hozzá tartozó View is automatikusan létrejön és frissül. Ez a két komponens szoros kapcsolatban áll, így amikor a ViewModel módosul, a hozzá kapcsolódó View is frissül [20].

A második kérdés, hogy miért képes a SelectedViewModel minden ViewModel-t kezelni? Erre a válasz az, hogy a különböző ViewModel-ek egy közös őssztályból,

úgynevezett BaseViewModel-ből származnak. Ez a közös ősosztály biztosítja a közös felületet, amely lehetővé teszi, hogy a SelectedViewModel egységes módon kezelje az összes alosztályt. Így a SelectedViewModel képes bármilyen típusú ViewModel-t kezelni, mivel mindegyik azonos alapfelületen épül fel. Ezt nevezzük polimorfizmusnak [15].

A „Home” menü opcióját elsősorban különböző hirdetések megjelenítésére tervezem. Ezen keresztül lehetne elérni a böngészőben futó híreket, valamint különféle ajánlatokkal kapcsolatos értesítéseket. A jövőben ezt a funkciót bővíteni szeretném, hogy még több hasznos információ és lehetőség érhető legyen el ezen a felületen.

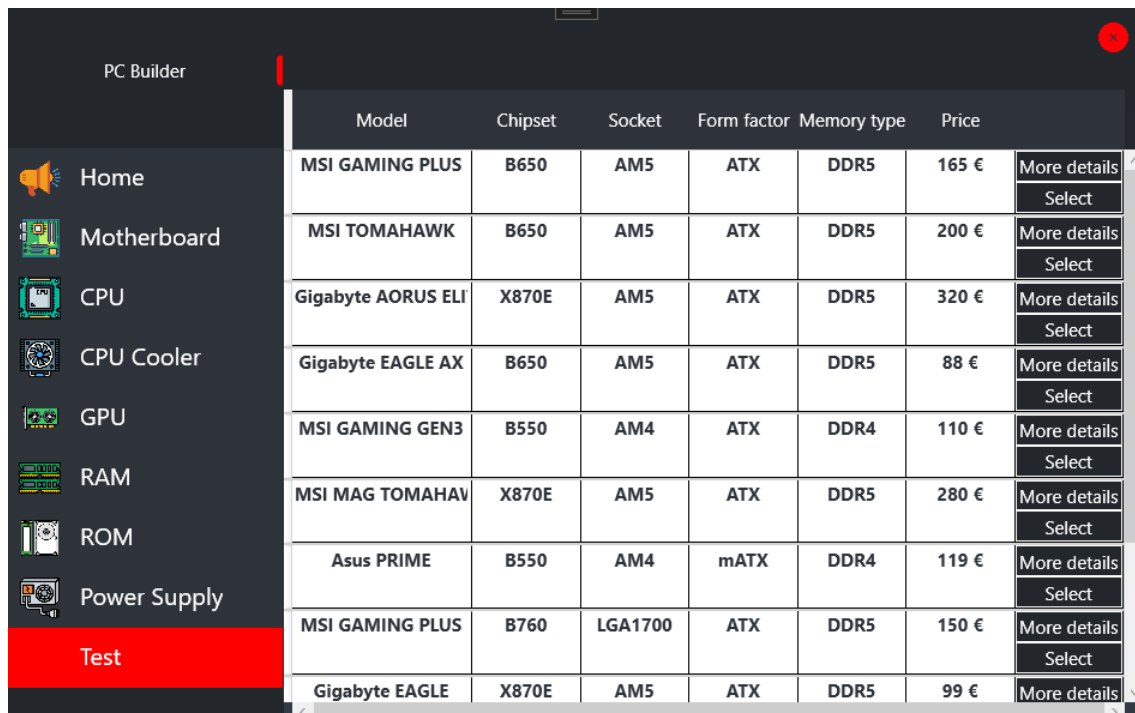
3.6.2. Listázásra használt nézet. Példa: MotherboardView

The image shows a screenshot of a web application interface for displaying motherboard data. It features a table with a dark theme. The table has a header row with the following columns: Model, Chipset, Socket, Form factor, Memory type, and Price. The body of the table is currently empty, showing only a vertical scrollbar on the left side. The table is enclosed in a dark border, and there is a small white icon on the left side of the table's vertical scrollbar.

Model	Chipset	Socket	Form factor	Memory type	Price
-------	---------	--------	-------------	-------------	-------

3.7. ábra. Maga a listázásra használt View.

A felhasználói élmény javítása érdekében a különböző alkatrészek egy DataGrid-ben jelennek meg, amely soronként listázza azokat. Az oszlopok alapértelmezés szerint a legfontosabb tulajdonságokat tartalmazzák, de amennyiben részletesebb információra van szükség, minden sor végén megtalálható egy „More details” gomb. Erre kattintva egy új ablak nyílik meg, amely a megfelelő View-val rendelkezik, és ott minden elérhető adat megjelenik az adatbázisból. Ezen kívül található egy „Select” gomb is, amellyel az adott komponenst kiválaszthatjuk. A kiválasztást követően a segédablak frissül, és megjeleníti az alkatrész nevét, valamint legfontosabb tulajdonságait. Ezenfelül a DataGrid egyik alapvető funkciója, hogy lehetőséget biztosít az oszlopok szélességének dinamikus módosítására, így a felhasználók igényeik szerint testre szabhatják az adatmegjelenítést. Ezen kívül, a gombok fölé húzva az egeret, egy vizuális effekt aktiválódik,



3.8. ábra. A főoldal kinézete kiválasztás után.

amely a Trigger segítségével van megvalósítva, így biztosítva a felhasználói interakciók vizuális visszajelzését [6].

3.7. Kódrészlet. A gombok a DataGrid sorban a kiválasztott adat megjelenítésére és kiválasztására szolgálnak.

```

1 <StackPanel>
2     <Button x:Name="BtnOpen" CommandParameter="{Binding.}"
3         Command="{Binding DataContext.SelectViewCommand,
4             ↪RelativeSource={RelativeSource AncestorType=
5             ↪UserControl}}">
6         Content="More details"
7         Style="{StaticResource GridDetailsButtonStyle}">
8     </Button>
9     <Button x:Name="BtnSelect" Content="Select"
10         CommandParameter="{Binding DataContext, RelativeSource={
11             ↪RelativeSource AncestorType=DataGridRow}}">
12         Command="{Binding DataContext.SelectPartCommand,
13             ↪RelativeSource={RelativeSource AncestorType=Window}}">
14         Style="{StaticResource GridSelectButtonStyle}" />
15 </StackPanel>

```

A fenti kód két gombot tartalmaz, amelyek a DataGrid sorában találhatóak, és a felhasználói interakcióval kapcsolatos funkciókat biztosítanak.

BtnOpen: Ez a gomb a kiválasztott elem részletes megjelenítését szolgálja. A gomb a SelectViewCommand parancsot hajtja végre, amelynek paramétere az aktuális

adatkontextus (a sor adatmodellje). A `CommandParameter` a gomb által kezelt adatot adja át [20].

BtnSelect: Ez a gomb a kiválasztott alkatrész adatainak rögzítésére szolgál. Az adatátvitelhez szükséges `CommandParameter` itt az aktuális sor adatkontextusát adja át, amely az adott alkatrész részletes adatait tartalmazza. A gomb a `SelectPartCommand` parancsot hajtja végre, amely a kiválasztott alkatrész további feldolgozását biztosítja [8].

RelativeSource attribútumok segítségével a parancsok és paraméterek az XAML hierarchiában történő megfelelő leképzésére van lehetőség, így a gombok megfelelően kapcsolódnak a szülőkontrollokhoz és azok adatkontextusaihoz [20].

A `MotherboardViewModel` osztály a `BaseViewModel` osztályból származik, és a Motherboard adatok kezelésére szolgál egy `ObservableCollection` segítségével. Az osztály különböző parancsokat (`ICommand`) és aszinkron adatlekérést valósít meg [20]. Az osztály feladata, hogy a felhasználói felület számára a szükséges adatokat biztosítsa és kezelje.

Propertyk

Motherboards: Kulcsfontosságú a megjelenítés szempontjából, ugyanis ezen keresztül a felhasználói felület folyamatosan frissíthető, ha a lista tartalma megváltozik. Ezt az `ObservableCollection()` [17] által tudja biztosítani.

3.8. Kódrészlet. Itt dől el, hogy melyik View kerül megjelenítésre.

```
1 private ObservableCollection<Motherboard> motherboardList = new
   ↳ ObservableCollection<Motherboard>();
2
3 public ObservableCollection<Motherboard> Motherboards
4 {
5     get { return motherboardList; }
6     set { motherboardList = value; }
7 }
```

SelectViewCommand: egy parancs, amely a felhasználó által választott komponens bővebb leírására szolgáló ablakot nyitja meg a megfelelő View-val. Példányosítása a `MotherboardViewModel` konstruktorában kerül sor.

SelectPartCommand: egy parancs, amely a felhasználó által választott komponenset behelyezi abba az `IConfig` példányba, ami később kompatibilitási tesztelésre kerül, illetve a segédablakban megjelenteti a kiválasztott alkatrészt.

Metódus

GetDatas(): Az GetDatas metódus aszinkron [16] módon lekéri az adatokat egy HTTP-kérelmen keresztül. A válasz JSON formátumban érkezik, amelyet a Structure típusba deszerializálunk. A deszerializálás során a motherboards, m2s és usbHeaders adatokat feldolgozzuk, és minden alaplapot (Motherboard) hozzáadunk a Motherboards listához. Ha hiba történik a lekérés közben, különböző hibakezelések történnek. Az egész program futás közben csak itt fog kommunikálni a szerverrel. Minden további műveletet az itt szerzett adatokkal hajtunk végre, figyelembe véve az erőforrások hatékony kezelését és védelmét. A menüből való kiválasztáskor azonnal megtörténik a lefutása, ugyanis a konstruktorban meg van hívva.

3.9. Kódrészlet. Itt kerül lekérdezésre az adat az adatbázisból a GetDatas()-ban.

```
1 HttpClient client = new HttpClient();
2 var response = await client.GetStringAsync("http://localhost:3000/
    ↪motherboards");
3 var option = new JsonSerializerOptions { PropertyNameCaseInsensitive =
    ↪ true };
4 Structure structure = JsonSerializer.Deserialize<Structure>(response,
    ↪ option);
```

Belső Osztály

Structure: ez az osztály tárolja a válaszként kapott JSON adatokat, amelyek lehetővé teszik a Motherboard, USBHeader és M2 típusú adatok megfelelő kezelését. Mivel az adatbázisunk 1:N kapcsolatú relációval rendelkezik, a későbbiekben ezen adatokat a megfelelő paraméterek figyelembevételével összekapcsoljuk, és az így kapott struktúrát felhasználjuk a további műveletekhez. Az osztály tehát az adat integrációját biztosítja, hogy a különböző típusú entitások összekapcsolásra kerüljenek a megfelelő logika szerint. Az adatokat a megfelelő mezőhöz a „JsonPropertyName(„,„)” akcióval tudjuk hozzárendelni a JSON válaszból.

SelectViewCommand

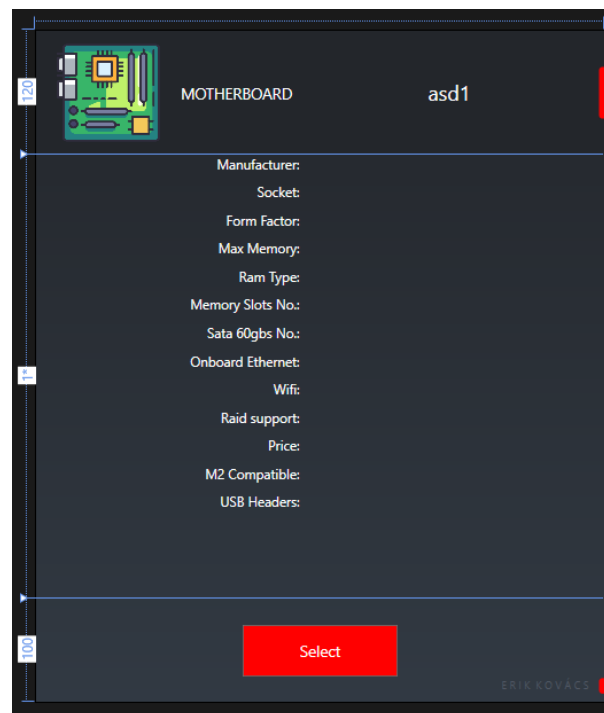
A SelectViewCommand pontosan úgy működik, mint a főoldalon található View frissítő Command, de néhány extra funkcióval is rendelkezik. A működése során egy Switch [5] szerkezetet használ, amely ellenőrzi, hogy melyik alkatrészcsoporthoz van szó, és az alapján létrehozza a megfelelő View-t.

Amikor a View elkészítésre kerül, a kiválasztott komponenst, amely a listában szerepel, átadja a View-hoz, hogy annak részletes adatai megjelenjenek. A végrehajtás során egy új ablak jön létre, és a ContentControl-be a Switch által kijelölt View-t ágyazza be, lehetővé téve ezzel a részletes információk megjelenítését a felhasználó számára.

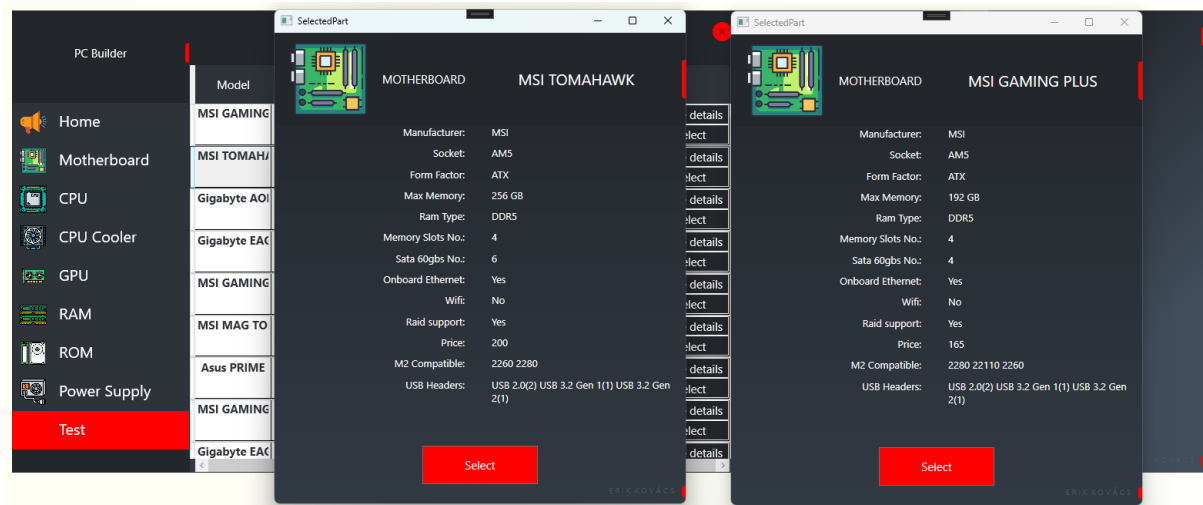
3.10. Kódrészlet. Az új ablak és View létrehozásának módja SelectViewCommand-ban.

```
1 private System.Windows.Controls.UserControl CreateView(IComputerPart  
  ↪viewName)  
2 {  
3     switch (viewName.Name())  
4     {  
5         case "Motherboard":  
6             return new SelectedMotherboardView(viewName);  
7         case "CPU":  
8             return new SelectedCPUView(viewName);  
9         ...  
10        default:  
11            return null;  
12    }  
13 }  
14 public void Execute(object parameter)  
15 {  
16     var selectedView = CreateView(parameter as IComputerPart);  
17     SelectedPart selectedPartWindow = new SelectedPart();  
18     selectedPartWindow.Content = selectedView;  
19     selectedPartWindow.Show();  
20 }
```

3.6.3. Kiválasztott alkatrész részletes megjelenítése. Példa: SelectedMotherboardView



3.9. ábra. Maga a kiválasztott alaplaphoz használt View.



3.10. ábra. Több komponens is megnyitható egyszerre.

A SelectedMotherboardView nézet a kiválasztott alaplaphoz részletes adatainak megjelenítésére szolgál. A szerkezete Grid-ből és StackPanel-ből épül fel, amelyek biztosítják a rugalmas és jól strukturált elrendezést.

Felépítés:

Háttér és Színkezelés: A Grid háttérszínét egy LinearGradientBrush [20] biztosítja, amely a szürke árnyalatait tartalmazza, ezzel biztosítva a vizuális elválasztást és

a letisztult megjelenést. **Grid elrendezés:** Első sor a nézet tetején elhelyezett Stack-Panel tartalmazza az alaplap nevét, valamint egy ikonra mutató képet. Második sor az adatokat tartalmazó StackPanel-ek elrendezése két oszlopra osztva, amely az alkatrész különböző jellemzőit jeleníti meg, például a gyártót, foglalatot, formátumot, max. memória stb .. Harmadik sor az alján található Select gomb, amely az alkatrész kiválasztását indítja el.

3.6.4. Kiválasztott alkatrész ViewModel-je. Példa: Selected-MotherboardViewModel

A SelectedMotherboardViewModel osztály egy ViewModel, amely a kiválasztott alaplap adatainak kezelésére és megjelenítésére szolgál a megfelelő nézetben. Az osztály az IComputerPart típusú objektumot, pontosabban egy Motherboard példányt dolgoz fel. IComputerPart egy közös felületet biztosít az alkatrészeknek. Később még kifejtésre fog kerülni.

Funkcionalitás:

SelectedMotherboard Property: Az SelectedMotherboard egy Motherboard típusú változó, amely az aktuálisan kiválasztott alaplapot reprezentálja. Az érték beállításakor az OnPropertyChanged metódus hívódik meg, hogy értesítse a felhasználói felületet az adatváltozásról. Ez biztosítja, hogy amikor az alaplap adataink módosulnak, az UI frissüljön.

SelectPartCommand: a parancs az adatkezelést irányítja. Az alkatrész kiválasztásához szükséges logikát hajtja végre (beállítja az adott alkatrészt a konfigurációba, amit később tesztelni fogunk), ugyanazt a kódsort használjuk mint a lista nézetben.

```
1 public void Execute(object parameter)
2 {
3     var viewModel = MainWindowViewModel.viewModel;
4     var configViewModel = ConfigWindowViewModel.viewModel;
5     switch (parameter)
6     {
7         case Motherboard motherboard:
8             viewModel.SetMotherboard(motherboard);
9             configViewModel.MotherboardModel = motherboard.Model + "
              ↳ + motherboard.Chipset + "
              ↳ " + motherboard.Socket + "
              ↳ " + motherboard.Ram_type;
10            break;
11        ...
12        default:
13            break;
14    }
15 }
```

Az egyik kulcsfontosságú elem a `viewModel` változó, amely a főoldal `ViewModel`-jére való hivatkozást tárolja. Ennek szerepe kiemelten fontos, hiszen minden kiválasztott alkatrészt a főoldal `ViewModel`-je fog kezelni, és ezen keresztül történik a konfiguráció összeállítása is. A `Command` ezen a kapcsolaton keresztül éri el és módosítja a megfelelő adatokat, ezért szükséges, hogy minden alkatrésztípus aktuálisan kiválasztott példánya ide kerüljön mentésre. A `viewModel` tehát gyakorlatilag egy adatfolyosót biztosít az egyes komponensnézetek és a fő nézetmodell között, lehetővé téve az adatok egységes és hatékony áramlását [8].

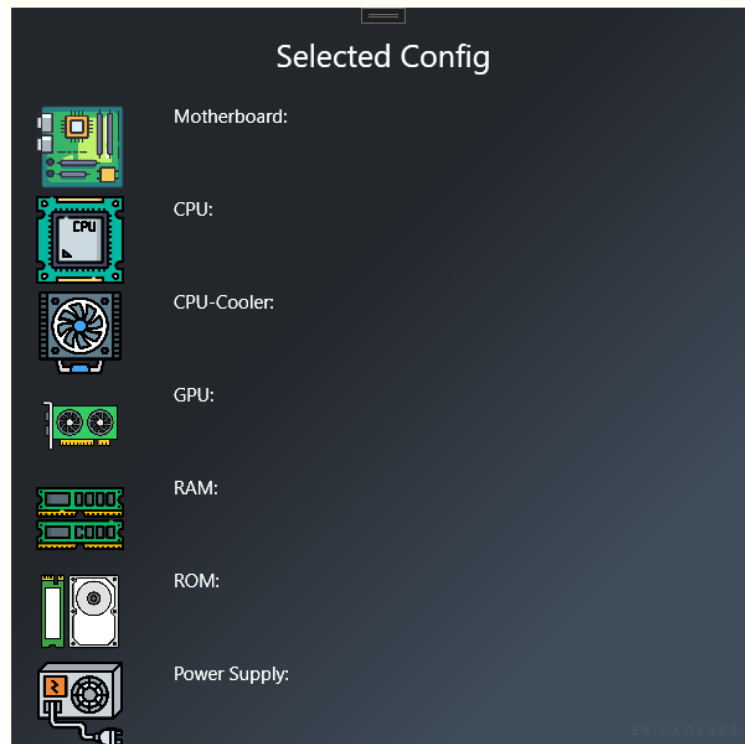
A `configViewModel` szintén hasonló, központi szerepet tölt be az alkalmazás működésében. Ez a változó biztosítja a kapcsolatot a segédablak – ahol az egyes alkatrészek adatai jelennek meg – és a teljes konfigurációs nézetmodell között. Mivel a segédablakban lévő UI-elemek, például a `TextBlock`-ok értékei dinamikusan frissülnek a kiválasztott komponens alapján, ezek módosítása is ezen a `configViewModel`-en keresztül történik. Ez lehetővé teszi, hogy a kiválasztás pillanatában az adott alkatrész adatai azonnal megjelenjenek és szinkronban legyenek a konfigurációs állapottal. A `configViewModel` kezdeti állapota természetesen üres, hiszen a felhasználó még nem választott komponenst – az értékek csak a kiválasztási művelet után töltődnek be. Ez a megközelítés biztosítja az adatok konzisztens és jól strukturált kezelését a különböző nézetrétegek között. Pontosan ez indokolja azt, hogy a főoldal és a segédablak statikus legyen [8].

A `SelectedMotherboardViewModel` osztály biztosítja a megfelelő adatokat a `SelectedMotherboard` és egyéb kapcsolódó mezők révén, amelyek a felhasználói felületen dinamikusan megjelennek a megfelelő `TextBlock` elemekben.

3.6.5. Segédablak

A fő ablak mellett megjelenik látszatra összekapcsolódva egymással. Egy teljesen új ablak. Bezárni nincs lehetőség, együtt indul a programmal és együtt záródik be a fő kikapcsoló gombbal [6]. Egy összefoglaló ablak, amelyben a felhasználó által kiválasztott alkatrészek adatai jelennek meg egy jól strukturált, látványos felületen. Az ablak célja, hogy valós időben tükrözze a konfiguráció aktuális állapotát a `ConfigViewModel` által biztosított adatkötéseken keresztül.

Az ablak `WindowStyle=„None”` és `AllowsTransparency=„True”` beállításai lehetővé teszik az egyedi megjelenést, azaz az ablaknak nincs szabványos kerete vagy címsora, így teljes mértékben testreszabható. A háttér egy átmenetes sötét dizájnt használ (`LinearGradientBrush`), amely egységes arculatot biztosít az alkalmazás többi nézetével [6]. A `Grid` konténer nyolc `RowDefinition` sorra van felosztva, minden sor egy-egy hardverkomponens (pl. alaplap, CPU, RAM) megjelenítésére szolgál. Minden sorban egy `StackPanel` van elhelyezve vízszintes orientációval, amely tartalmazza:



3.11. ábra. Alap helyzetben a segédablak.

- Az adott alkatrész ikonját (Image)
- Egy statikus címkét (pl . Motherboard:, GPU:)
- Egy TextBlock-ot, amely dinamikusan az adott modell nevét jeleníti meg az aktuális ViewModel értéke alapján (Text=„Binding XYZModel”)

A TextWrapping=„Wrap” opció biztosítja, hogy hosszabb modellek is jól olvashatóan, törve jelenjenek meg.

ViewModel

Minden alkatrész csoporthoz tartozik egy property, amely segítségével frissítjük az adatokat, amik OnPropertyChanged() tulajdonsággalv annak ellátva a UI frissítése érdekében.

3.11. Kódrészlet. Egyik property.

```

1 public string RAMModel
2 {
3     get { return ramModel; }
4     set { ramModel = value; OnPropertyChanged(nameof(RAMModel)); }
5 }

```

Az ablak működésének egyik legfontosabb eleme a ViewModel által implementált INotifyPropertyChanged interfész [16]. Ez biztosítja a kommunikációt a View és a

ViewModel között: ha bármelyik tulajdonság értéke megváltozik, a PropertyChanged esemény hatására a WPF automatikusan frissíti a felhasználói felületet. Ez a mechanizmus kulcsszerepet játszik az adatkötésben (data binding), és egyben az MVVM architektúra alapköve, mivel lehetővé teszi a felület és a mögöttes logika közötti kohéziót minden egyes View és ViewModel között.

3.12. Kódrészlet. INotifyPropertyChanged implementáció.

```
1 public event PropertyChangedEventHandler PropertyChanged;
2 protected void OnPropertyChanged(string propertyName)
3 {
4     PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(
5         ↪propertyName));
6 }
```

3.6.6. Teszt funkció

A tesztelési funkciót egy külön osztály és egy hozzá tartozó Command osztály valósítja meg. A teszt gomb megnyomásakor a Command aktiválódik, amely példányosítja a tesztelésért felelős osztályt, majd lefuttatja annak logikáját. A tesztelés során az osztály egy listába gyűjti a konfigurációban található problémákat vagy inkompatibilitásokat, és ezt a listát visszaadja a Command számára [8]. Amennyiben a hibalista nem üres, úgy egy hibaüzeneteket megjelenítő View kerül megnyitásra egy felugró (popup) ablakban. Ha viszont nem található hiba a konfigurációban, a rendszer egy sikeres visszajelzést nyújtó View-t jelenít meg.

TestCommand osztály

A TestCommand osztály az MVVM architektúrában a tesztelési folyamatot indítja el, amikor a felhasználó rákattint a „Teszt” gombra. Az osztály az ICommand interfészt valósítja meg, így közvetlenül köthető gombhoz vagy más vezérlőelemhez a View-ban. Az Execute metódus tartalmazza a parancs logikáját. Itt történik meg egy Compatibility_Checker objektum példányosítása, amely egy külön osztályba szervezve végzi el a tényleges kompatibilitási vizsgálatot.

3.13. Kódrészlet. Az Execute alapvető beállítása.

```
1 var checker = new PC_Builder.Checker.Compatibility_Checker();
2 IComputer computer = parameter as IComputer;
3 List<string> errors = checker.GetErrors();
```

A felhasználó által összeállított konfiguráció – amit a parancs paraméterként kap meg – egyenként átadásra kerül a vizsgáló metódusoknak, például a processzor, az alaplap, a memória vagy a tápegység. Minden ilyen vizsgálat során a rendszer ellenőrzi, hogy az adott komponens illeszkedik-e a többihez, illetve megfelel-e az általános

elvárásoknak. Pontosan ezt hívják Visitor Design Pattern vagy Látogató Tervezési Mintának.

3.14. Kódrészlet. Maga a Visitor belépő része.

```
1 checker.VisitMotherboard(computer.motherboard);
2 checker.VisitROM(computer.rom);
3 checker.VisitRAM(computer.ram);
4 checker.VisitGPU(computer.gpu);
5 checker.VisitCPU(computer.cpu);
6 checker.VisitCPUCooler(computer.cpu_Cooler);
7 checker.VisitPS(computer.power_supply);
```

A vizsgálat során fellépő problémák gyűjtésére egy hibalista szolgál, amelyet a tesztelő objektum tölt fel. Ha a lista nem üres, vagyis valamilyen hiba merült fel a konfigurációban, akkor a parancs egy hiba-ablakot jelenít meg, amely részletesen felsorolja az észlelt problémákat. Ezzel szemben, ha a tesztelés során minden rendben található, akkor egy másik, pozitív eredményt közlő ablak jelenik meg, amely arról tájékoztatja a felhasználót, hogy a konfiguráció működőképes és minden komponens kompatibilis egymással.

Külön figyelmet kap a hibakezelés is. Amennyiben a felhasználó nem adott meg minden szükséges alkatrészt, és emiatt a kompatibilitási vizsgálat nem hajtható végre teljes körűen, a rendszer kivételt dob, amelyet a parancs megfelelően lekezel. Ilyenkor egy külön erre a célra létrehozott hibaüzenet ablak jelenik meg, amely közli a felhasználóval, hogy hiányzó komponens miatt a tesztelés nem végezhető el.

3.15. Kódrészlet. Maga a Visitor belépő része.

```
1 catch (EmptySelectedPartException ex)
2 {
3     ErrorWindow errorWindow = new ErrorWindow(ex.Message);
4     errorWindow.Show();
5 }
```

3.6.7. Teszt funkció

A tesztelési funkciót egy külön osztály és egy hozzá tartozó Command osztály valósítja meg. A teszt gomb megnyomásakor a Command aktiválódik, amely példányosítja a tesztelésért felelős osztályt, majd lefuttatja annak logikáját. A tesztelés során az osztály egy listába gyűjti a konfigurációban található problémákat vagy inkompatibilitásokat, és ezt a listát visszaadja a Command számára. Amennyiben a hibalista nem üres, úgy egy hibaüzeneteket megjelenítő View kerül megnyitásra egy felugró (popup) ablakban. Ha viszont nem található hiba a konfigurációban, a rendszer egy sikeres visszajelzést nyújtó View-t jelenít meg.

TestCommand osztály

A TestCommand osztály az MVVM architektúrában a tesztelési folyamatot indítja el, amikor a felhasználó rákattint a „Teszt” gombra. Az osztály az ICommand interfészt valósítja meg, így közvetlenül köthető gombhoz vagy más vezérlőelemhez a View-ban. Az Execute metódus tartalmazza a parancs logikáját. Itt történik meg egy Compatibility_Checker objektum példányosítása, amely egy külön osztályba szervezve végzi el a tényleges kompatibilitási vizsgálatot.

3.16. Kódrészlet. Az Execute alapvető beállítása.

```
1 var checker = new PC_Builder.Checker.Compatibility_Checker();
2 IComputer computer = parameter as IComputer;
3 List<string> errors = checker.GetErrors();
```

A felhasználó által összeállított konfiguráció – amit a parancs paraméterként kap meg – egyenként átadásra kerül a vizsgáló metódusoknak, például a processzor, az alaplap, a memória vagy a tápegység. Minden ilyen vizsgálat során a rendszer ellenőrzi, hogy az adott komponens illeszkedik-e a többihez, illetve megfelel-e az általános elvárásoknak. Pontosan ezt hívják Visitor Design Pattern vagy Látogató Tervezési Mintának[7].

3.17. Kódrészlet. Maga a Visitor belépő része.

```
1 checker.VisitMotherboard(computer.motherboard);
2 checker.VisitROM(computer.rom);
3 checker.VisitRAM(computer.ram);
4 checker.VisitGPU(computer.gpu);
5 checker.VisitCPU(computer.cpu);
6 checker.VisitCPUCooler(computer.cpu_Cooler);
7 checker.VisitPS(computer.power_supply);
```

A vizsgálat során fellépő problémák gyűjtésére egy hibalista szolgál, amelyet a tesztelő objektum tölt fel. Ha a lista nem üres, vagyis valamilyen hiba merült fel a konfigurációban, akkor a parancs egy hiba-ablakot jelenít meg, amely részletesen felsorolja az észlelt problémákat. Ezzel szemben, ha a tesztelés során minden rendben található, akkor egy másik, pozitív eredményt közlő ablak jelenik meg, amely arról tájékoztatja a felhasználót, hogy a konfiguráció működőképes és minden komponens kompatibilis egymással.

Külön figyelmet kap a hibakezelés is. Amennyiben a felhasználó nem adott meg minden szükséges alkatrészt, és emiatt a kompatibilitási vizsgálat nem hajtható végre teljes körűen, a rendszer kivételt dob, amelyet a parancs megfelelően lekezel. Ilyenkor egy külön erre a célra létrehozott hibaüzenet ablak jelenik meg, amely közli a felhasználóval, hogy hiányzó komponens miatt a tesztelés nem végezhető el.

3.18. Kódrészlet. Maga a Visitor belépő része.

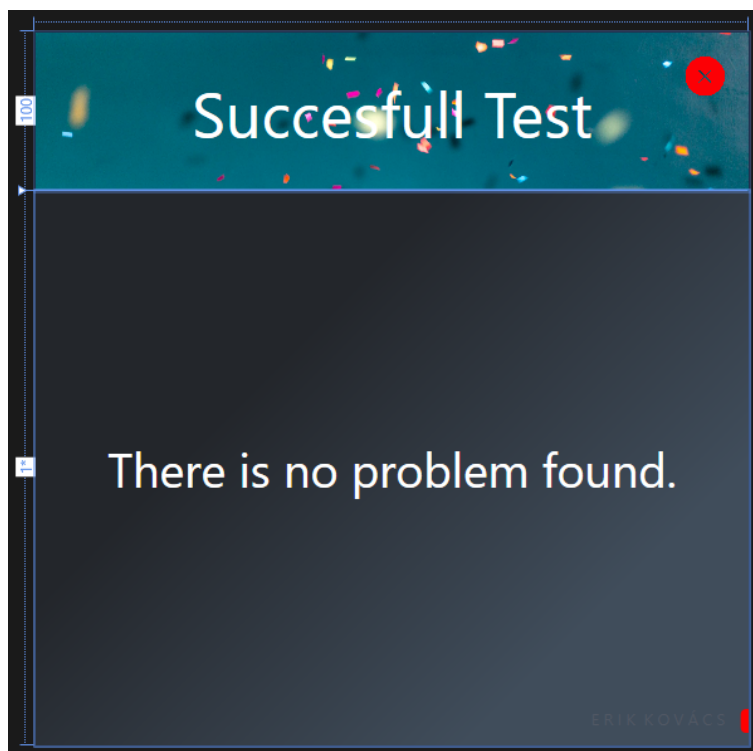
```

1 catch (EmptySelectedPartException ex)
2 {
3     ErrorWindow errorWindow = new ErrorWindow(ex.Message);
4     errorWindow.Show();
5 }

```

3.6.8. Az eredményt kimutató ablak

Szakmai szempontból ez a megoldás egy tartalomorientált, dinamikusan változó felületű popup ablakot valósít meg, amely a ContentControl elem segítségével jeleníti meg a megfelelő nézetet. Az ablak kezdetben üres, de a futásidejű logika — konkrétan a tesztelés eredménye — határozza meg, hogy melyik View kerül bele a vezérlőbe. Amennyiben a konfigurációban nem található hiba, úgy a sikeres tesztet reprezentáló View jelenik meg. Ha azonban a kompatibilitás-ellenőrzés során bármilyen problémát észlel a rendszer, a hibákat tartalmazó lista alapján a sikertelen teszt eredményét megjelenítő View töltődik be. Ebben az esetben a hibalista minden egyes eleme egy-egy teljes mondatként jelenik meg, világosan és részletesen kommunikálva a felhasználó számára, hogy mely komponensek nem illeszkednek egymáshoz a konfigurációban.



3.12. ábra. A sikert jelentő View.

A view háttere egy lineáris színátmenet (LinearGradientBrush), amely sötét, modern tónusokkal emeli ki a központi tartalmat. A felület két fő sorra van bontva: egy

felső szekcióra, amely a fejléct és a vizuális visszajelzést tartalmazza, és egy alsó szekcióra, ahol a részletes üzenet olvasható.

A felső szekcióban egy háttérkép (OKBG.jpg) található, amely vizuálisan is alátámasztja a sikeres eredményt. Ezen a rétegen középre igazítva jelenik meg a „Successful Test” felirat, elegáns, fehér betűszínnel és nagy méretben. Ezt egészíti ki egy jobb felső sarokban elhelyezkedő, stílusozott Bezárás gomb, amely az ablak bezárásáért felelős.

Az alsó szekcióban egy középre igazított üzenet olvasható: „There is no problem found.” — ez ad szöveges megerősítést a felhasználónak arról, hogy a rendszer minden ellenőrzött komponense kompatibilis egymással.



3.13. ábra. A sikertelen tesztet jelentő View.

A FailedTestResultPopupWindowView a WPF-alapú alkalmazás azon nézete, amely akkor kerül megjelenítésre, ha a konfigurációs teszt során inkompatibilitások vagy más hibák merülnek fel. Ez a vizuális komponens egy dinamikusan betöltődő View, amely a fő PopupWindow tartalmát képezi, amennyiben a konfiguráció nem felel meg a kompatibilitási elvárásoknak.

A nézet kialakítása hasonló a sikeres teszt View szerkezetéhez, ezzel biztosítva az egységes felhasználói élményt. A háttér itt is egy sötét tónusú, két színből álló lineáris színátmenet, amely vizuálisan támogatja a hibák súlyosságát és kontrasztosan emeli ki a tartalmat.

A felső szekcióban egy figyelemfelkeltő háttérkép (failedTestBG.png) kapott helyet, amelyre rákerül a középre igazított „Test Failed” felirat, élénk piros színnel és nagy méretben. A jobb felső sarokban elhelyezett bezárás gomb (BtnClose) lehetőséget biztosít

az ablak bezárására, ugyanazzal a stílus-definícióval, mint a sikeres nézet esetében.

Az alsó szekcióban történik a konkrét hibák megjelenítése. Az itt található TextBlock az MVVM minta szerint a ViewModel-ből érkező errors tulajdonsághoz van kötve (Binding errors). Ez a tulajdonság tartalmazza az összes hibát szöveges formában, melyeket a kompatibilitás-ellenőrző osztály gyűjtött össze. A hibák olvashatóságát a TextWrapping és a nagy betűméret segíti, így a felhasználó gyorsan és világosan tájékozódhat a problémákról.

Az alsó sarokban ismét megjelenik a diszkrét fejlesztői aláírás, illetve a piros dekoratív sáv, amely az egész alkalmazás vizuális identitásának része.

Ezek a nézetek a ContentControl segítségével dinamikusan kerülnek betöltésre a popup ablakba, a kompatibilitásellenőrzést végző logika döntése alapján. A sikeres vagy sikertelen eredményt vizuálisan kiemelve és egyszerű, jól érthető üzenettel támogatja a felhasználói élményt.

4. fejezet

Tesztelés

A tesztelési folyamat kiemelkedő szerepet játszott az alkalmazás fejlesztése során, mivel biztosítani kellett a rendszer stabilitását, megbízhatóságát és megfelelő működését. A projekt során manuális és automatizált tesztelési módszereket egyaránt alkalmaztam annak érdekében, hogy az egyes funkciók megfelelően működjenek, és a rendszer minden követelménynek eleget tegyen. A manuális tesztelés lehetőséget biztosított arra, hogy a felhasználói felületet, az adatkezelést és az egyes funkciókat részletesen és célzottan ellenőrizsem, míg az automatizált tesztelés gyorsabbá és hatékonyabbá tette a tesztelési folyamatot, biztosítva, hogy a kód minden módosítása után az alkalmazás továbbra is megfelelően működjön.

4.1. Manuális tesztelés

A fejlesztési folyamat során folyamatosan manuális tesztek végeztem, amelyek során alaposan átvizsgáltam az alkalmazás különböző aspektusait. A manuális tesztelés egyik legfontosabb területe a felhasználói felület ellenőrzése volt, amely során minden egyes nézetet, vezérlőelemet és interakciót teszteltem annak érdekében, hogy azok megfelelően jelenjenek meg és működjenek. Fontos volt, hogy a felhasználók számára intuitív és könnyen kezelhető felületet biztosítsak, ezért különös figyelmet fordítottam az UI-elemek megfelelő viselkedésére és visszajelzéseire.

A funkcionalitás tesztelése során minden egyes új funkciót kipróbáltam, hogy megbizonyosodjak annak helyes működéséről. Minden fejlesztési fázis után manuálisan ellenőriztem az alkalmazás különböző részeit, hogy az új komponensek zökkenőmentesen illeszkedjenek a rendszerbe, és ne okozzanak hibákat a meglévő funkciókban.

4.1.1. Tesztelési jegyzőkönyv - Főablak

Adatok

Tesztelő: Kovács Erik

Operációs rendszer: Windows 11 22H2 (64 bit)

Környezet: Visual Studio 2022

Tesztesetek

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test1_01	Az ablak és kinézete	Az ablak megjelenik	OK
Test1_02	Az ablak és kinézete	Az ablakra vonatkozó stílus megjelenik	OK
Test1_03	Az ablak és kinézete	A cím megjelenik a megfelelő helyen és stílusban	OK
Test1_04	Az ablak és kinézete	A cím szegélye és stílusa megfelelően megjelenik	OK
Test1_05	Az ablak és kinézete	A menü megjelenik	OK
Test1_06	Az ablak és kinézete	A menü elemei megjelennek	OK
Test1_07	Az ablak és kinézete	Az menü elemeinek a képe megjelenik	OK
Test1_08	Az ablak és kinézete	Az menü stílusa megfelelően megjelenik	OK
Test1_09	Az ablak és kinézete	Az bezáró gomb megjelenik	OK
Test1_10	Az ablak és kinézete	A bezáró gomb stílusa megfelelően megjelenik	OK
Test1_11	Az ablak és kinézete	Az ablak indulásakor a Home töltődik be	OK
Test1_12	Az ablak és kinézete	A menü felett az egér pontterré változik	OK
Test1_13	Az ablak és kinézete	A bezáró gomb felett az egér pointerré változik	OK
Test1_14	A menü elem viselkedése	A menü elem fölé húzva az egeret megváltozik az elem háttere	OK
Test1_15	Az ablak és kinézete	A menü felett az egér pontterré változik	OK

Test1_16	Az ablak és kinézete	A jobb alsó sarokban megjelenik a készítő neve	OK
Test1_17	Az ablak és kinézete	A jobb alsó sarokban megjelenik a szegély a megfelelő stílusban	OK
Test1_18	A menü elem viselkedése	Az egyik menü elemre kattintva (kivéve a teszt) megjelenik a megfelelő alkatrész lista	OK

Adatok

Tesztelő: Fábián Dániel (Pont Systems KFT.)

Operációs rendszer: Windows 11 24H2 (64 bit)

Környezet: Visual Studio 2019

Tesztesetek

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test2_01	Az ablak és kinézete	Az ablak megjelenik	OK
Test2_02	Az ablak és kinézete	Az ablakra vonatkozó stílus megjelenik	OK
Test2_03	Az ablak és kinézete	A cím megjelenik a megfelelő helyen és stílusban	OK
Test2_04	Az ablak és kinézete	A cím szegélye és stílusa megfelelően megjelenik	OK
Test2_05	Az ablak és kinézete	A menü megjelenik	OK
Test2_06	Az ablak és kinézete	A menü elemei megjelennek	OK
Test2_07	Az ablak és kinézete	Az menü elemeinek a képe megjelenik	OK
Test2_08	Az ablak és kinézete	Az menü stílusa megfelelően megjelenik	OK
Test2_09	Az ablak és kinézete	Az bezáró gomb megjelenik	OK

Test2_10	Az ablak és kinézete	A bezáró gomb stílusa megfelelően megjelenik	OK
Test2_11	Az ablak és kinézete	Az ablak indulásakor a Home töltődik be	OK
Test2_12	Az ablak és kinézete	A menü felett az egér pointerre változik	OK
Test2_13	Az ablak és kinézete	A bezáró gomb felett az egér pointerre változik	OK
Test2_14	A menü elem viselkedése	A menü elem fölé húzva az egeret megváltozik az elem háttere	OK
Test2_15	Az ablak és kinézete	A menü felett az egér pointerre változik	OK
Test2_16	Az ablak és kinézete	A jobb alsó sarokban megjelenik a készítő neve	OK
Test2_17	Az ablak és kinézete	A jobb alsó sarokban megjelenik a szegély a megfelelő stílusban	OK
Test2_18	A menü elem viselkedése	Az egyik menü elemre kattintva (kivéve a teszt) megjelenik a megfelelő alkatrész lista	OK

Adatok

Tesztelő: Jovanovic Andrija (OMV Wien - IT Grupp)

Operációs rendszer: Windows 11 25H2 (64 bit)

Környezet: Visual Studio 2022

Tesztesetek

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test3_01	Az ablak és kinézete	Az ablak megjelenik	OK
Test3_02	Az ablak és kinézete	Az ablakra vonatkozó stílus megjelenik	OK
Test3_03	Az ablak és kinézete	A cím megjelenik a megfelelő helyen és stílusban	OK
Test3_04	Az ablak és kinézete	A cím szegélye és stílusa megfelelően megjelenik	OK
Test3_05	Az ablak és kinézete	A menü megjelenik	OK
Test3_06	Az ablak és kinézete	A menü elemei megjelennek	OK
Test3_07	Az ablak és kinézete	Az menü elemeinek a képe megjelenik	OK
Test3_08	Az ablak és kinézete	Az menü stílusa megfelelően megjelenik	OK
Test3_09	Az ablak és kinézete	Az bezáró gomb megjelenik	OK
Test3_10	Az ablak és kinézete	A bezáró gomb stílusa megfelelően megjelenik	OK
Test3_11	Az ablak és kinézete	Az ablak indulásakor a Home töltődik be	OK
Test3_12	Az ablak és kinézete	A menü felett az egér pointerre változik	OK
Test3_13	Az ablak és kinézete	A bezáró gomb felett az egér pointerre változik	OK
Test3_14	A menü elem viselkedése	A menü elem fölé húzva az egeret megváltozik az elem háttere	OK
Test3_15	Az ablak és kinézete	A menü felett az egér pointerre változik	OK
Test3_16	Az ablak és kinézete	A jobb alsó sarokban megjelenik a készítő neve	OK

Test3_17	Az ablak és kinézete	A jobb alsó sarokban megjelenik a szegély a megfelelő stílusban	OK
Test3_18	A menü elem viselkedése	Az egyik menü elemre kattintva (kivéve a teszt) megjelenik a megfelelő alkatrész lista	OK

4.1.2. Tesztelési jegyzőkönyv - Lista nézet

Adatok

Tesztelő: Kovács Erik

Operációs rendszer: Windows 11 22H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test1_19	A lista nézet kinézete	A nézet a megfelelő stílusban jelenik meg	OK
Test1_20	A lista nézet kinézete	A nézetben a fejléc megfelelően megjelenik	OK
Test1_21	A lista nézet kinézete	A megfelelő alkatrész lista kilistázódik	OK
Test1_22	A lista nézet kinézete	A lista elemek jobb oldalán megjelenik a két gomb a megfelelő stílusban	OK
Test1_23	A lista nézet kinézete	A lista elemeinek akciógombjai fölé húzva az egeret pointerre változik és a háttér is átváltozik	OK
Test1_24	A lista nézet kinézete	Listába kattintás során az adott elem aljának színe megváltozik	OK
Test1_25	A lista nézet kinézete	A listában az elemek tulajdonságai a megfelelő cellában jelennek meg	OK

Test1_26	A „More details” gomb működése	Megjelent az új ablak	OK
Test1_27	A „More details” gomb működése	A megfelelő view jelenik meg az új ablakban	OK
Test1_52	A lista „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

Adatok

Tesztelő: Fábián Dániel (Pont Systems KFT.)

Operációs rendszer: Windows 11 24H2 (64 bit)

Környezet: Visual Studio 2019

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test2_19	A lista nézet kinézete	A nézet a megfelelő stílusban jelenik meg	OK
Test2_20	A lista nézet kinézete	A nézetben a fejléc megfelelően megjelenik	OK
Test2_21	A lista nézet kinézete	A megfelelő alkatrész lista kilistázódik	OK
Test2_22	A lista nézet kinézete	A lista elemek jobb oldalán megjelenik a két gomb a megfelelő stílusban	OK
Test2_23	A lista nézet kinézete	A lista elemeinek akciógombjai fölé húzva az egeret pointerre változik és a háttér is átváltozik	OK
Test2_24	A lista nézet kinézete	Listába kattintás során az adott elem aljának színe megváltozik	OK
Test2_25	A lista nézet kinézete	A listában az elemek tulajdonságai a megfelelő cellában jelennek meg	OK
Test2_26	A „More details” gomb működése	Megjelent az új ablak	OK
Test2_27	A „More details” gomb működése	A megfelelő view jelenik meg az új ablakban	OK
Test2_52	A lista „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

Adatok

Tesztelő: Jovanovic Andrija (OMV Wien - IT Grupp)

Operációs rendszer: Windows 11 25H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test3_19	A lista nézet kinézete	A nézet a megfelelő stílusban jelenik meg	OK
Test3_20	A lista nézet kinézete	A nézetben a fejléc megfelelően megjelenik	OK
Test3_21	A lista nézet kinézete	A megfelelő alkatrész lista kilistázódik	OK
Test3_22	A lista nézet kinézete	A lista elemek jobb oldalán megjelenik a két gomb a megfelelő stílusban	OK
Test3_23	A lista nézet kinézete	A lista elemeinek akciógombjai fölé húzva az egeret pointerre változik és a háttér is átváltozik	OK
Test3_24	A lista nézet kinézete	Listába kattintás során az adott elem aljának színe megváltozik	OK
Test3_25	A lista nézet kinézete	A listában az elemek tulajdonságai a megfelelő cellában jelennek meg	OK
Test3_26	A „More details” gomb működése	Megjelent az új ablak	OK
Test3_27	A „More details” gomb működése	A megfelelő view jelenik meg az új ablakban	OK
Test3_52	A lista „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

4.1.3. Tesztelési jegyzőkönyv - Részletező ablak

Adatok

Tesztelő: Kovács Erik

Operációs rendszer: Windows 11 22H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test1_28	A részletező ablak kinézete	A megfelelő fejléc ikon került megjelenítésre	OK
Test1_29	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test1_30	A részletező ablak kinézete	A fejlécben megjelenik a piros szegély	OK
Test1_31	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test1_32	A részletező ablak kinézete	Az ablak a megfelelő stílusban jelent meg	OK
Test1_33	A részletező ablak kinézete	Alul megjelenik a „Select” gomb	OK
Test1_34	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test1_35	A részletező ablak „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

Adatok

Tesztelő: Fábián Dániel (Pont Systems KFT.)

Operációs rendszer: Windows 11 24H2 (64 bit)

Környezet: Visual Studio 2019

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test2_28	A részletező ablak kinézete	A megfelelő fejléc ikon került megjelenítésre	OK
Test2_29	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test2_30	A részletező ablak kinézete	A fejlécben megjelenik a piros szegély	OK
Test2_31	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test2_32	A részletező ablak kinézete	Az ablak a megfelelő stílusban jelent meg	OK
Test2_33	A részletező ablak kinézete	Alul megjelenik a „Select” gomb	OK
Test2_34	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test2_35	A részletező ablak „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

Adatok

Tesztelő: Jovanovic Andrija (OMV Wien - IT Grupp)

Operációs rendszer: Windows 11 25H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test3_28	A részletező ablak kinézete	A megfelelő fejléc ikon került megjelenítésre	OK
Test3_29	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test3_30	A részletező ablak kinézete	A fejlécben megjelenik a piros szegély	OK
Test3_31	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test3_32	A részletező ablak kinézete	Az ablak a megfelelő stílusban jelent meg	OK
Test3_33	A részletező ablak kinézete	Alul megjelenik a „Select” gomb	OK
Test3_34	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test3_35	A részletező ablak „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

4.1.4. Tesztelési jegyzőkönyv - Részletező ablak

Adatok

Tesztelő: Kovács Erik

Operációs rendszer: Windows 11 22H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test1_28	A részletező ablak kinézete	A megfelelő fejléc ikon került megjelenítésre	OK
Test1_29	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test1_30	A részletező ablak kinézete	A fejlécben megjelenik a piros szegély	OK
Test1_31	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test1_32	A részletező ablak kinézete	Az ablak a megfelelő stílusban jelent meg	OK
Test1_33	A részletező ablak kinézete	Alul megjelenik a „Select” gomb	OK
Test1_34	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test1_35	A részletező ablak „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

Adatok

Tesztelő: Fábián Dániel (Pont Systems KFT.)

Operációs rendszer: Windows 11 24H2 (64 bit)

Környezet: Visual Studio 2019

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test2_28	A részletező ablak kinézete	A megfelelő fejléc ikon került megjelenítésre	OK
Test2_29	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test2_30	A részletező ablak kinézete	A fejlécben megjelenik a piros szegély	OK
Test2_31	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test2_32	A részletező ablak kinézete	Az ablak a megfelelő stílusban jelent meg	OK
Test2_33	A részletező ablak kinézete	Alul megjelenik a „Select” gomb	OK
Test2_34	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test2_35	A részletező ablak „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

Adatok

Tesztelő: Jovanovic Andrija (OMV Wien - IT Grupp)

Operációs rendszer: Windows 11 25H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test3_28	A részletező ablak kinézete	A megfelelő fejléc ikon került megjelenítésre	OK
Test3_29	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test3_30	A részletező ablak kinézete	A fejlécben megjelenik a piros szegély	OK
Test3_31	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test3_32	A részletező ablak kinézete	Az ablak a megfelelő stílusban jelent meg	OK
Test3_33	A részletező ablak kinézete	Alul megjelenik a „Select” gomb	OK
Test3_34	A részletező ablak kinézete	A fejlécben megjelenik a kiválasztott modell neve	OK
Test3_35	A részletező ablak „Select” gomb működése	A kiválasztott komponens bekerült a konfigurációba és megjelenik a segédablakban	OK

4.1.5. Tesztelési jegyzőkönyv - Teszt funkció

Adatok

Tesztelő: Kovács Erik

Operációs rendszer: Windows 11 22H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test1_43	A Teszt gomb funkciója megfelelő konfiguráció esetén	Megjelenik a popup ablak a „sikeres” view használatával	OK
Test1_44	A Teszt gomb funkciója nem megfelelő konfiguráció esetén	Megjelenik a popup ablak a sikertelen view használatával	OK
Test1_45	A Teszt gomb funkciója, hiányos konfiguráció esetén	Megjelenik a popup ablak a sikertelen view használatával	OK
Test1_46	A Teszt funkció lefutása után megjelenik az ablak a képernyő közepén	A teszt funkció lefutása után megjelenik az ablak közepén	OK

Adatok

Tesztelő: Fábián Dániel (Pont Systems KFT.)

Operációs rendszer: Windows 11 24H2 (64 bit)

Környezet: Visual Studio 2019

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test2_43	A Teszt gomb funkciója megfelelő konfiguráció esetén	Megjelenik a popup ablak a „sikeres” view használatával	OK
Test2_44	A Teszt gomb funkciója nem megfelelő konfiguráció esetén	Megjelenik a popup ablak a sikertelen view használatával	OK
Test2_45	A Teszt gomb funkciója, hiányos konfiguráció esetén	Megjelenik a popup ablak a sikertelen view használatával	OK
Test2_46	A Teszt funkció lefutása után megjelenik az ablak a képernyő közepén	A teszt funkció lefutása után megjelenik az ablak közepén	OK

Adatok

Tesztelő: Jovanovic Andrija (OMV Wien - IT Grupp)

Operációs rendszer: Windows 11 25H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test3_43	A Teszt gomb funkciója megfelelő konfiguráció esetén	Megjelenik a popup ablak a „sikeres” view használatával	OK
Test3_44	A Teszt gomb funkciója nem megfelelő konfiguráció esetén	Megjelenik a popup ablak a sikertelen view használatával	OK
Test3_45	A Teszt gomb funkciója, hiányos konfiguráció esetén	Megjelenik a popup ablak a sikertelen view használatával	OK
Test3_46	A Teszt funkció lefutása után megjelenik az ablak a képernyő közepén	A teszt funkció lefutása után megjelenik az ablak közepén	OK

4.1.6. Tesztelési jegyzőkönyv - Eredményt jelző felugró ablak

Adatok

Tesztelő: Kovács Erik

Operációs rendszer: Windows 11 22H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test1_47	A sikeres teszt eredményt jelző ablak kinézete	Az ablak a megfelelő stílussal jelenik meg	OK
Test1_48	A sikeres teszt eredményét jelző ablak kinézete	A view a megfelelő stílusban jelenik meg	OK
Test1_49	A sikeres teszt eredményét jelző ablak kinézete	A view a megfelelő szöveget tartalmazza	OK
Test1_47	A sikertelen teszt eredményt jelző ablak kinézete	Az ablak a megfelelő stílussal jelenik meg	OK
Test1_48	A sikertelen teszt eredményét jelző ablak kinézete	A view a megfelelő stílusban jelenik meg	OK
Test1_49	A sikertelen teszt eredményét jelző ablak kinézete	A view a megfelelő szöveget tartalmazza	OK
Test1_50	A sikertelen teszt eredményét jelző ablak bezáró gombjának működése	Bezárja a felugró ablakot	OK
Test1_51	A sikeres teszt eredményét jelző ablak bezáró gombjának működése	Bezárja a felugró ablakot	OK

Adatok

Tesztelő: Fábián Dániel (Pont Systems KFT.)

Operációs rendszer: Windows 11 24H2 (64 bit)

Környezet: Visual Studio 2019

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test2_47	A sikeres teszt eredményt jelző ablak kinézete	Az ablak a megfelelő stílussal jelenik meg	OK
Test2_48	A sikeres teszt eredményét jelző ablak kinézete	A view a megfelelő stílusban jelenik meg	OK
Test2_49	A sikeres teszt eredményét jelző ablak kinézete	A view a megfelelő szöveget tartalmazza	OK
Test2_47	A sikertelen teszt eredményt jelző ablak kinézete	Az ablak a megfelelő stílussal jelenik meg	OK
Test2_48	A sikertelen teszt eredményét jelző ablak kinézete	A view a megfelelő stílusban jelenik meg	OK
Test2_49	A sikertelen teszt eredményét jelző ablak kinézete	A view a megfelelő szöveget tartalmazza	OK
Test2_50	A sikertelen teszt eredményét jelző ablak bezáró gombjának működése	Bezárja a felugró ablakot	OK
Test2_51	A sikeres teszt eredményét jelző ablak bezáró gombjának működése	Bezárja a felugró ablakot	OK

Adatok

Tesztelő: Jovanovic Andrija (OMV Wien - IT Grupp)

Operációs rendszer: Windows 11 25H2 (64 bit)

Környezet: Visual Studio 2022

Azonosító	Teszt leírása	Elvárt viselkedés	Eredmény
Test3_47	A sikeres teszt eredményt jelző ablak kinézete	Az ablak a megfelelő stílussal jelenik meg	OK
Test3_48	A sikeres teszt eredményét jelző ablak kinézete	A view a megfelelő stílusban jelenik meg	OK
Test3_49	A sikeres teszt eredményét jelző ablak kinézete	A view a megfelelő szöveget tartalmazza	OK
Test3_47	A sikertelen teszt eredményt jelző ablak kinézete	Az ablak a megfelelő stílussal jelenik meg	OK
Test3_48	A sikertelen teszt eredményét jelző ablak kinézete	A view a megfelelő stílusban jelenik meg	OK
Test3_49	A sikertelen teszt eredményét jelző ablak kinézete	A view a megfelelő szöveget tartalmazza	OK
Test3_50	A sikertelen teszt eredményét jelző ablak bezáró gombjának működése	Bezárja a felugró ablakot	OK
Test3_51	A sikeres teszt eredményét jelző ablak bezáró gombjának működése	Bezárja a felugró ablakot	OK

4.2. Automatizált tesztelés

A manuális tesztelés mellett automatizált teszteléseket is végeztem annak érdekében, hogy hosszú távon biztosítsam a rendszer stabilitását, valamint gyors és pontos visszajelzést kapjak minden egyes fejlesztési ciklus után. Az automatizált tesztelés lehetővé tette, hogy a rendszer különböző moduljai folyamatos megfigyelés alatt álljanak, és a kód módosításai ne eredményezzenek váratlan hibákat vagy regressziókat.

A legfontosabb automatizált tesztelési módszer, amelyet alkalmaztam, az egységtesztelés volt. Ennek során a rendszer egyes komponenseit izoláltan teszteltem, biztosítva, hogy minden funkció önállóan is a várt módon viselkedjen. Az egységtesztek megírásához a MSTest keretrendszert használtam, amely szorosan integrálódik a Visual Studio fejlesztői környezetbe, és lehetővé teszi a tesztek gyors futtatását, valamint az eredmények áttekinthető kiértékelését. A tesztesztályokban külön figyelmet fordítottam a kivételkezelés, bemeneti validáció és üzleti logika helyes működésének ellenőrzésére.

Az egységtesztelés mellett integrációs tesztek is implementáltak, amelyek célja az volt, hogy ellenőrizsem az alkalmazás különböző rétegei – például az adatbázis, a backend és a frontend – közötti kommunikációt. Ezek során teszteltem az API-k megfelelő működését, az adatok pontos továbbítását, valamint az esetleges adattranszformációk helyességét. Az integrációs tesztekhez Postman és Swagger eszközöket használtam manuális validálásra, automatizált környezetben pedig REST-assured és xUnit alapú szkriptekkel vizsgáltam a RESTful szolgáltatások működését.

A tesztelési folyamat részeként a Continuous Integration (CI) szemléletet is alkalmaztam: a tesztek minden kódmódosítás után automatikusan lefutottak a Git verziókezelő rendszerrel integrált GitHub Actions segítségével, így azonnali visszacsatolást kaptam az esetleges hibákról. Ez nagyban hozzájárult a gyors hibajavításhoz és a stabil build-ek fenntartásához.

A fejlesztési folyamat végére a manuális és automatizált tesztelések kombinálásával sikerült egy stabil, megbízható és jól működő alkalmazást létrehozni, amely megfelelt mind a funkcionális, mind a nem-funkcionális elvárásoknak.

5. fejezet

Befejezés

A számítógép-összeállító és kompatibilitás-ellenőrző alkalmazás fejlesztése során olyan technológiákat és eszközöket választottam, amelyek egyszerre biztosítják a hatékony működést, a jó teljesítményt és a felhasználóbarát élményt. A Node.js és az Express.js kombinációja lehetővé tette egy gyors, aszinkron backend kialakítását, míg a MySQL és a PHPMyAdmin segítségével stabil adatkezelés valósult meg. A REST API-n keresztül történő kommunikáció leegyszerűsítette az adatok átvitelét a szerver és a kliens között, a C# és WPF alapú frontend pedig reszponzív, letisztult felhasználói felületet eredményezett.

A projekt megvalósításának célja az volt, hogy egy olyan eszközt hozzak létre, amely segíti a felhasználókat a számítógép-alkatrészek kiválasztásában, és képes automatikusan ellenőrizni azok kompatibilitását. A rendszer használata nemcsak egyszerűbbé, hanem biztonságosabbá is teszi a PC-építés folyamatát, különösen azok számára, akik kevésbé jártasak a hardveres kérdésekben.

Úgy gondolom, hogy a projekt nemcsak technológiai szempontból szolgált tanulságokkal, hanem a felhasználói igények megértésében is segített. A jövőben a rendszer továbbfejleszthető lenne például dinamikus konfiguráció-ajánlásokkal, árfigyelő funkciókkal, vagy akár közösségi megosztási lehetőségekkel is.

Ez a fejlesztés nem csupán szakmai kihívás volt, hanem egy lehetőség is arra, hogy egy gyakorlati problémára hasznos, működő megoldást kínáljak.

Irodalomjegyzék

- [1] Cantelon M., Harter M., Holowaychuk T., Rajlich N.: *Node.js in Action*, Shelter Island, 2013, NY: Manning Publications.
- [2] Evan Hahn: *Express in Action: Writing, building, and testing Node.js applications*, Shelter Island, 2016., Manning Publications
- [3] Kozmajer Viktor: *PHP és MySQL az alapoktól*, Budapest, 2011, BBS-INFO Kiadó.
- [4] Mark Masse: *REST API Design Rulebook*, Sebastopol, 2011, O'Reilly Media
- [5] Mark J. Price: *C# 10 and .NET 6 – Modern Cross-Platform Development*, Birmingham, 2021, Packt Publishing
- [6] Adam Nathan: *Windows Presentation Foundation Unleashed*, Indianapolis, 2013, Sams Publishing
- [7] Eric Freeman és Elisabeth Robson: *Head First Design Patterns: A Brain-Friendly Guide*, Sebastopolban, 2004, O'Reilly Media
- [8] Josh Smith: *Advanced MVVM*, 2010, Lulu.com
- [9] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides *Design Patterns: Elements of Reusable Object-Oriented Software*, Boston, 1994, Addison-Wesley Professional
- [10] Alistair Cockburn: *Writing Effective Use Cases*, Boston, 2000, Addison-Wesley
- [11] Németh Tamás: *Git & GitHub Visual Guide: The Hands-On Manual for Complete Beginners*, Budapest, 2023, Self-published
- [12] Paul DuBois: *MySQL Cookbook: Solutions for Database Developers and Administrators*, Sebastopol, 2009, O'Reilly Media
- [13] C.J. Date: *SQL and Relational Theory: How to Write Accurate SQL Code*, Sebastopol, 2009, O'Reilly Media
- [14] Seyed M. M. R.: *Learning MySQL*, Sebastopol, 2017, O'Reilly Media

- [15] Joseph Albahari, Ben Albahari: *C# 9.0 in a Nutshell: A Desktop Quick Reference*, Sebastopol, 2020, O'Reilly Media
- [16] Jon Skeet: *C# in Depth*, Shelter Island, 2019, Manning Publications
- [17] Anders Hejlsberg, Scott Wiltamuth, Philips D. LeBlanc: *The C# Programming Language*, Boston, 2003, Addison-Wesley
- [18] Jason Boyd: *Build Your Own PC: The Ultimate Guide to Building Your Own Computer*, Charleston, 2021, Self-published
- [19] Robin Heydon: *PC Hardware in a Nutshell*, Sebastopol, 2004, O'Reilly Media
- [20] Chris Sells, Adam Nathan: *Professional WPF Programming: .NET Development with Windows Presentation Foundation*, Hoboken, 2006, Wiley

Nyilatkozat

Alulírott *Kovács Erik*, büntetőjogi felelősségem tudatában kijelentem, hogy az általam benyújtott, *Számítógép építő többretegű alkalmazás* című szakdolgozat önálló szellemi termékem. Amennyiben mások munkáját felhasználtam, azokra megfelelően hivatkozom, beleértve a nyomtatott és az internetes forrásokat is.

Aláírással igazolom, hogy az elektronikusan feltöltött és a papíralapú szakdolgozatom formai és tartalmi szempontból mindenben megegyezik.

Eger, 2025. április 14.

aláírás

Kovács Erik